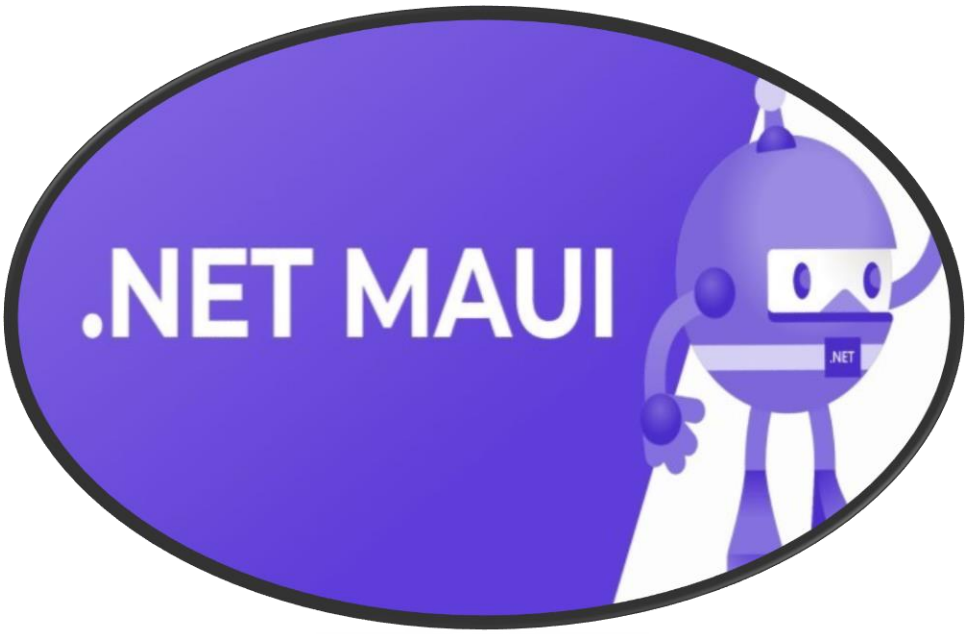


Cross-platform apps bouwen in .NET MAUI

ROC van Twente

Opleiding Software Development
Leerjaar 3

Marcel Roesink

Inhoudsopgave

Hoofdstuk 1: Inleiding tot .NET MAUI 6
1.1 Wat is .NET MAUI? 6
1.2 Waarom .NET MAUI? 6
1.3 Wat heb je nodig om aan de slag te gaan? 6
1.4 Vergelijkbare ontwikkeltools 7
Oefeningen 7
Conclusie 9
Hoofdstuk 2: Configuratie en Installatie 10
2.1 Installatie van Visual Studio 2022 10
Stappen: 10
2.2 Installatie van de .NET SDK 10
Stappen: 11
2.3 Configuratie van Android SDK en Emulator 11
Stappen: 11
2.4 Windows Ontwikkelomgeving Instellen 11
2.5 Basiskennis van C# en XAML 12
Oefeningen 12
Conclusie 13
Hoofdstuk 3: Je eerste .NET MAUI Applicatie 14
3.1 Aanmaken van een .NET MAUI Project 14
Stappen: 14
3.2 Verkennen van de Projectstructuur 14
3.2 1 Belangrijke onderdelen: 14
3.3 Aanpassen van de "Hello World" Applicatie 15
Stappen: 15
3.4 Builden en Uitvoeren van de Applicatie 16
Stappen: 16
Oefeningen 17
Conclusie 19
Inleveropdracht 1 20
Hoofdstuk 4: Werken met layouts 21
4.1 StackLayout 21
4.1.1 Horizontal StackLayout: 21

4.1.2 Vertical StackLayout: 21
4.2 Grid 22
4.3 AbsoluteLayout 22
4.4 FlexLayout 22
4.4.1 Horizontal FlexLayout: 22
4.4.2 Vertical FlexLayout: 23
Oefeningen 23
Video tutorials 23
Hoofdstuk 5: Toevoegen van Functionaliteit aan je .NET MAUI Applicatie 24
5.1 Werken met Verschillende Controls 24
Voorbeeldcontrols: 24
Stappen: 24
5.2 Data Binding 27
5.2.1 Eenvoudige Data Binding: 27
5.3 Hanteren van Gebruikersinvoer 29
Voorbeeld: 29
Oefeningen 30
Video tutorials 31
Conclusie 31
Inleveropdracht 2 32
Functionele eisen: 32
Niet-functionele eisen: 32
Inleveropdracht 3 33
Functionele eisen: 33
Inleveropdracht 4 34
Functionele eisen: 34
Hoofdstuk 6: Geavanceerde Functionaliteiten en Gebruikerservaring 35
6.1 Navigatie tussen Pagina's 35
6.2 Gebruik van Ingebouwde Functionaliteiten van het Apparaat 36
6.3 Implementeren van Animaties en Visuele Effecten 37
Oefeningen 37
Conclusie 38
Inleveropdracht 5 39
Hoofdstuk 7: Verschillende Soorten Pagina's in .NET MAUI 41
7.1 ContentPage 41
Voorbeeld: Een Eenvoudige ContentPage 41

Gebruik in XAML 42
7.2 FlyoutPage 42
Voorbeeld: Een Eenvoudige FlyoutPage 43
7.3 TabbedPage 44
Gebruik in XAML 45
7.4 NavigationPage 45
7.4.1 Voorbeeld: Een Eenvoudige NavigationPage 45
Oefeningen 46
Conclusie 47
Inleveropdracht 6 48
Hoofdstuk 8: Werken met externe data en API's 50
8.1 Het Ophalen van data van een Web-API 50
8.2 Werken met JSON Data 53
8.3 Afhandelen van Fouten en Netwerkproblemen 53
Voorbeeld: 55
Oefeningen 56
Conclusie 57
Inleveropdracht 7 58
Keuzeopdracht 7.1 – maximaal 30 van 50 punten 58
Keuzeopdracht 7.2 – maximaal 30 van 50 punten 58
Keuzeopdracht 7.3 – maximaal 40 van 50 punten 59
Keuzeopdracht 7.4 – maximaal 50 van 50 punten 59
Hoofdstuk 9: Internationale Ondersteuning en Lokalisatie 60
9.1 Wat is Lokalisatie? 60
9.2 Implementeren van Lokalisatie in .NET MAUI 60
9.2.1 Voorbereiden van je Project 60
9.2.2 Configureren van Lokalisatie in je Applicatie 61
9.2.3 Dynamische Lokalisatie 62
Oefeningen 63
Conclusie 64
Inleveropdacht 8 65
Hoofdstuk 10: Integratie van Externe Bibliotheken 67
10.1 Wat zijn Externe Bibliotheken? 67
10.2 Installeren van Externe Bibliotheken 67
10.2.1 Installatie via Visual Studio 67
10.2.2 Installatie via de .NET CLI 67

10.3 Voorbeeld: Integratie van SQLite voor Lokale Dataopslag 67
10.3.1 Installeren van het SQLite NuGet-pakket 68
10.3.2 Configureren van SQLite in je Project 68
Oefeningen 71
Conclusie 72
Inleveropdracht 9 73
Hoofdstuk 11: Push Notifications in .NET MAUI 75
Oefeningen 79
Conclusie 79
Inleveropdracht 10 80
Hoofdstuk 12: Eindproject 82
Projectopdracht: Een Complete .NET MAUI Applicatie 82
Stappen 82
Beoordelingsmatrix 84
Conclusie 86

Hoofdstuk 1: Inleiding tot .NET MAUI

1.1 Wat is .NET MAUI?

.NET MAUI (Multi-platform App UI) is een cross-platform framework van Microsoft dat is ontworpen om het ontwikkelen van applicaties voor verschillende besturingssystemen eenvoudiger en efficiënter te maken. Het stelt ontwikkelaars in staat om met één enkele codebase native applicaties te maken voor Windows, Android, iOS, en macOS. Hoewel we ons in dit werkboek richten op Windows en Android, biedt MAUI de flexibiliteit om je applicaties eenvoudig uit te breiden naar andere platforms als dat in de toekomst nodig is.

1.2 Waarom .NET MAUI?

1. **Cross-platform ontwikkeling**: Eén van de grootste voordelen van .NET MAUI is de mogelijkheid om één codebase te gebruiken om applicaties te ontwikkelen voor meerdere platforms. Dit bespaart tijd en moeite omdat je niet meerdere afzonderlijke projecten hoeft te onderhouden.
2. **Native prestaties**: MAUI-applicaties worden gecompileerd naar native code, wat zorgt voor betere prestaties in vergelijking met sommige andere cross-platform frameworks die gebruik maken van interpretatie of webtechnologieën.
3. **Gebruik van .NET en C#**: Als onderdeel van het .NET-ecosysteem maakt MAUI gebruik van C#, een krachtige en veelzijdige programmeertaal. Dit betekent dat je gebruik kunt maken van alle voordelen van het .NET-platform, zoals uitgebreide bibliotheken, sterke typecontrole en geavanceerde ontwikkeltools.
4. **Componenten en herbruikbaarheid**: MAUI maakt gebruik van XAML voor het definiëren van de gebruikersinterface, waardoor het eenvoudig is om UI-componenten te hergebruiken en consistente, aantrekkelijke interfaces te creëren.
5. **Ondersteuning van moderne ontwerppatronen**: Met MAUI kun je moderne ontwerppatronen zoals MVVM (Model-View-ViewModel) gemakkelijk toepassen, wat de ontwikkeling en onderhoud van je applicaties vereenvoudigt.

1.3 Wat heb je nodig om aan de slag te gaan?

1. **Visual Studio 2022**: Dit is de geïntegreerde ontwikkelomgeving (IDE) die je nodig hebt om .NET MAUI-applicaties te maken. Zorg ervoor dat je de juiste workloads installeert tijdens de installatie van Visual Studio:
    - .NET desktop development
    - Mobile development with .NET
2. **.NET SDK**: De .NET SDK moet geïnstalleerd zijn om .NET MAUI-projecten te kunnen aanmaken en bouwen. Dit wordt meestal automatisch beheerd door Visual Studio.
3. **Android SDK en Emulator**: Voor het ontwikkelen en testen van Android-applicaties heb je de Android SDK en een emulator nodig. Deze kunnen ook worden geïnstalleerd via Visual Studio.
4. **Windows 10 of hoger**: Voor de ontwikkeling van Windows-applicaties moet je op een Windows 10 of hoger systeem werken.

5. **Basiskennis van C# en XAML**: Hoewel dit werkboek je door de basis van MAUI zal leiden, is het handig als je al enige kennis hebt van C# en XAML. C# is de programmeertaal die wordt gebruikt voor de logica van je applicaties, terwijl XAML wordt gebruikt voor het definiëren van de gebruikersinterface.

1.4 Vergelijkbare ontwikkeltools

Er zijn verschillende andere frameworks en tools beschikbaar voor cross-platform ontwikkeling die vergelijkbare functionaliteit bieden als .NET MAUI:

1. **Flutter**: Ontwikkeld door Google, is Flutter een populair open-source framework voor het bouwen van natively gecompileerde applicaties voor mobiele, web- en desktopplatforms vanuit een enkele codebase. Het maakt gebruik van de programmeertaal Dart en biedt een rijke set aan widgets en tools.
2. **React Native**: Een framework van Facebook waarmee ontwikkelaars mobiele applicaties kunnen bouwen met JavaScript en React. Het biedt toegang tot native componenten en API's, waardoor de prestaties verbeteren en een native look en feel wordt geboden.
3. **Xamarin**: Ook van Microsoft en de voorganger van .NET MAUI, Xamarin stelt ontwikkelaars in staat om mobiele applicaties te bouwen met C# en .NET. Xamarin.Forms, onderdeel van Xamarin, is vergelijkbaar met MAUI en wordt gebruikt om cross-platform user interfaces te maken.
4. **Ionic**: Ionic is een open-source framework dat gebruik maakt van webtechnologieën zoals HTML, CSS en JavaScript om mobiele applicaties te ontwikkelen. Het kan samen met Angular, React, of Vue worden gebruikt en biedt een set aan UI-componenten die native-like ervaringen bieden.
5. **Unity**: Hoewel voornamelijk bekend voor gameontwikkeling, kan Unity ook worden gebruikt om non-game applicaties te ontwikkelen voor mobiele platforms. Het biedt krachtige grafische mogelijkheden en ondersteuning voor C#.

De reden dat we in dit werkboek .NET MAUI behandelen is dat het gebruik maakt van C# als programmeertaal en XAML als opmaaktaal. Beide zijn onderdeel van Visual Studio en je hebt in eerdere leerjaren al de nodige kennis opgedaan van C#.

Oefeningen

Oefening 1: Installatie van Visual Studio 2022

1. **Doel**: Installeer Visual Studio 2022 met de benodigde workloads.
2. **Stappen**:
   - Ga naar de Visual Studio website en download de Visual Studio 2022 Community Edition.
   - Tijdens de installatie, selecteer de volgende workloads:
     - .NET desktop development
     - Mobile development with .NET
   - Voltooi de installatie.
3. **Controle**: Open Visual Studio 2022 en ga naar "Help" > "About Microsoft Visual Studio" om te bevestigen dat de juiste versie en workloads zijn geïnstalleerd.

Oefening 2: Aanmaken van een .NET MAUI Project

1. **Doel**: Maak een eenvoudig .NET MAUI-project aan.
2. **Stappen**:
    - Open Visual Studio 2022.
    - Klik op "Create a new project".
    - Zoek en selecteer "MAUI App" en klik op "Next".
    - Geef je project een naam, bijvoorbeeld "HelloMaui" en klik op "Create".
    - Selecteer de standaardtemplate en klik op "Create".
    - **Controle**: Controleer of het project correct is aangemaakt door het te bouwen en uit te voeren op zowel Windows als de Android-emulator.

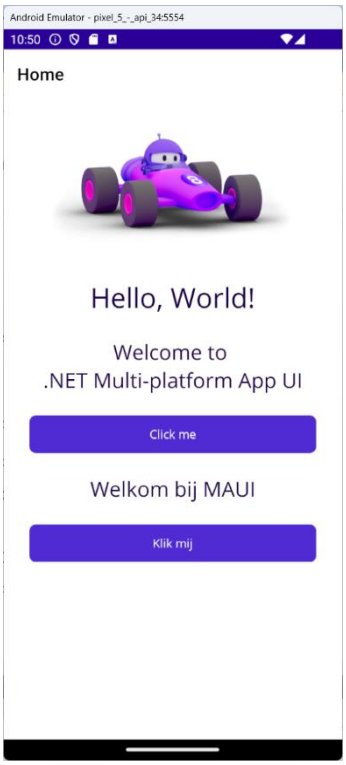
Oefening 3: Eerste UI-Wijzigingen

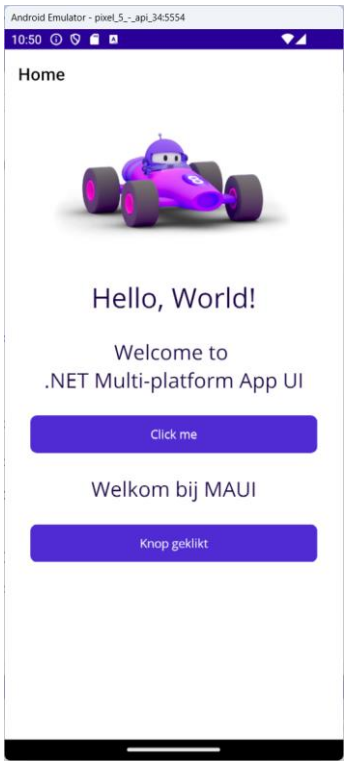
1. **Doel**: Pas de gebruikersinterface van je MAUI-app aan.
2. **Stappen**:

- Open het bestand MainPage.xaml in Visual Studio.
- Voeg een label en een knop toe aan de bestaande layout.
- Pas de tekst van het label en de knop aan, bijvoorbeeld: <Label Text="Welkom bij MAUI!" HorizontalOptions="Center" VerticalOptions="CenterAndExpand" />
- Voeg een klik-event toe aan de knop in MainPage.xaml.cs:

```
private void OnButtonClicked(object sender, EventArgs e)
{
    (sender as Button).Text = "Knop geklikt!";
}
```

3. **Controle**: Bouw en voer je project uit op zowel Windows als de Android-emulator om te controleren of de wijzigingen correct zijn toegepast en werken zoals verwacht. De volgende screenshots laten zien hoe de app er in Android uitziet.





Conclusie

Met .NET MAUI kun je efficiënte, native applicaties bouwen voor meerdere platforms met een enkele codebase. Dit werkboek zal je begeleiden door de basisprincipes van MAUI en je voorzien van praktische opdrachten om je vaardigheden te ontwikkelen. We zullen ons richten op het maken van Windows en Android apps, zodat je een sterke basis hebt om vanuit te werken. Laten we beginnen met de installatie en configuratie van je ontwikkelomgeving.

Hoofdstuk 2: Configuratie en Installatie

In dit hoofdstuk gaan we dieper in op de configuratie en installatie van de benodigde tools en SDK's voor het ontwikkelen van .NET MAUI-applicaties. We behandelen de stappen die nodig zijn om je ontwikkelomgeving volledig in te richten, zodat je klaar bent om je eerste MAUI-applicatie te maken.

2.1 Installatie van Visual Studio 2022

Visual Studio 2022 is de geïntegreerde ontwikkelomgeving (IDE) die je nodig hebt om .NET MAUI-applicaties te ontwikkelen. Volg de onderstaande stappen om Visual Studio 2022 te installeren en te configureren.

Stappen:

1. **Downloaden en installeren van Visual Studio 2022**:

- Ga naar de Visual Studio website en download de Community Edition van Visual Studio 2022.
- Voer het gedownloade installatieprogramma uit.

2. **Selecteren van workloads tijdens de installatie**:

- Tijdens de installatie wordt je gevraagd om workloads te selecteren. Kies de volgende workloads:
    - **.NET desktop development**: Dit is nodig voor het ontwikkelen van Windows-applicaties.
    - **Mobile development with .NET**: Dit is nodig voor het ontwikkelen van Android-applicaties.

3. **Voltooien van de installatie**:

- Klik op "Install" en wacht tot de installatie is voltooid. Dit kan enige tijd duren, afhankelijk van je internetverbinding en systeemprestaties.

4. **Controle**:

- Open Visual Studio 2022 na de installatie.
- Ga naar "Help" > "About Microsoft Visual Studio" om te bevestigen dat de juiste versie en workloads zijn geïnstalleerd.

2.2 Installatie van de .NET SDK

De .NET SDK is nodig om .NET MAUI-projecten te kunnen aanmaken en bouwen. Volg de onderstaande stappen om de .NET SDK te installeren.

Stappen:

1. **Downloaden en installeren van de .NET SDK**:

- Ga naar de downloadpagina van .NET en download de nieuwste versie van de .NET SDK.
- Voer het gedownloade installatieprogramma uit en volg de instructies op het scherm om de installatie te voltooien.

2. **Controle**:

- Open een terminal (Command Prompt, PowerShell, of Terminal op macOS).
- Voer het commando dotnet --version uit om te controleren of de .NET SDK correct is geïnstalleerd. Je zou een versie moeten zien die overeenkomt met de geïnstalleerde versie.

2.3 Configuratie van Android SDK en Emulator

Voor het ontwikkelen en testen van Android-applicaties heb je de Android SDK en een emulator nodig. Deze kunnen worden geïnstalleerd via Visual Studio.

Stappen:

1. **Android SDK en Emulator installeren**:

- Open Visual Studio 2022.
- Ga naar "Tools" > "Android" > "Android SDK Manager".
- Installeer de aanbevolen SDK-platforms en tools.
- Ga naar "Tools" > "Android" > "Android Device Manager".
- Maak een nieuwe emulator aan met de gewenste configuraties (bijv. Pixel 5, Android 11).

2. **Controle**:

- Start de Android Emulator via de "Android Device Manager" in Visual Studio.
- Controleer of de emulator correct opstart en functioneert.

2.4 Windows Ontwikkelomgeving Instellen

Voor de ontwikkeling van Windows-applicaties moet je op een Windows 10 of hoger systeem werken. Zorg ervoor dat je systeem up-to-date is met de nieuwste updates en patches.

Stappen:

1. **Controleren op updates**:

- Ga naar "Instellingen" > "Bijwerken en beveiliging" > "Windows Update" en klik op "Controleren op updates".
- Installeer alle beschikbare updates.

2. **Instellen van ontwikkelmodus**:

- Ga naar "Instellingen" > "Bijwerken en beveiliging" > "Voor ontwikkelaars".
- Schakel de ontwikkelmodus in om aanvullende ontwikkeltools en instellingen beschikbaar te maken.

2.5 Basiskennis van C# en XAML

Hoewel dit werkboek je door de basis van MAUI zal leiden, is het handig als je al enige kennis hebt van C# en XAML. Hier zijn enkele bronnen om je kennis op te frissen:

1. **C# Tutorials**:
   - De officiële C# tutorials van Microsoft bieden een uitstekende introductie tot de taal.
2. **XAML Tutorials**:
   - De officiële XAML tutorials van Microsoft bieden een overzicht van hoe XAML werkt en hoe je het kunt gebruiken om gebruikersinterfaces te definiëren.

Oefeningen

Oefening 4: Installatie van de .NET SDK

1. **Doel**: Installeer de .NET SDK op je systeem.
2. **Stappen**:

- Ga naar de downloadpagina van .NET.
- Download en installeer de nieuwste versie van de .NET SDK.
- Open een terminal en voer dotnet --version uit om de installatie te verifiëren.

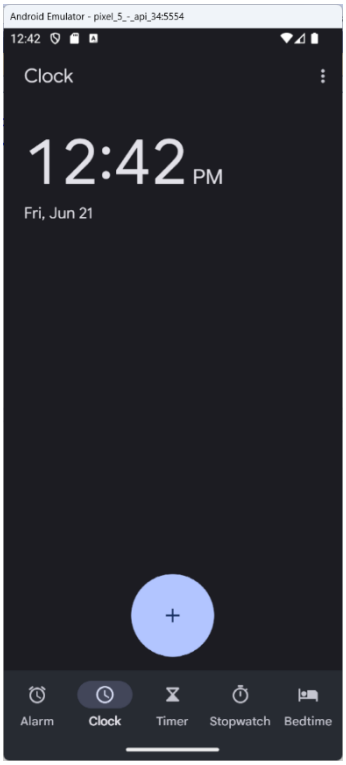
3. **Controle**: Zorg ervoor dat de juiste versie van de .NET SDK is geïnstalleerd door de uitvoer van `dotnet --version` te controleren.

Oefening 5: Configuratie van Android Emulator

1. **Doel**: Configureer en start een Android Emulator.
2. **Stappen**:

- Open Visual Studio 2022.
- Ga naar "Tools" > "Android" > "Android Device Manager".
- Maak een nieuwe emulator aan met de gewenste configuraties (bijv. Pixel 5, Android 11).
- Start de emulator.

3. **Controle**: Controleer of de emulator correct opstart en functioneert door een eenvoudige Android-app, bijvoorbeeld de klok, in de emulator te draaien.



Oefening 6: Instellen van de ontwikkelmodus in Windows

1. **Doel**: Schakel de ontwikkelmodus in op je Windows-systeem.
2. **Stappen**:

- Ga naar "Instellingen" > "Bijwerken en beveiliging" > "Voor ontwikkelaars".
- Schakel de ontwikkelmodus in.

3. **Controle**: Zorg ervoor dat de ontwikkelmodus is ingeschakeld door te controleren of de bijbehorende opties beschikbaar zijn in de instellingen.

Conclusie

Met deze configuratie- en installatie-instructies ben je klaar om te beginnen met het ontwikkelen van .NET MAUI-applicaties. In de volgende hoofdstukken zullen we dieper ingaan op het bouwen van je eerste applicatie, het aanpassen van de gebruikersinterface, en het toevoegen van functionaliteit. Laten we aan de slag gaan met het volgende hoofdstuk en je eerste .NET MAUI-applicatie maken!

Hoofdstuk 3: Je eerste .NET MAUI Applicatie

In dit hoofdstuk gaan we een eenvoudige .NET MAUI-applicatie maken. Dit zal je helpen vertrouwd te raken met de basisprincipes van .NET MAUI en de ontwikkelomgeving. We zullen een eenvoudige "Hello World" applicatie maken en deze aanpassen om je kennis van XAML en C# in de context van .NET MAUI te versterken.

3.1 Aanmaken van een .NET MAUI Project

We beginnen met het aanmaken van een nieuw .NET MAUI-project in Visual Studio 2022.

Stappen:

1. **Open Visual Studio 2022**:

- Start Visual Studio 2022 op je systeem.

2. **Maak een nieuw project aan**:

- Klik op "Create a new project".
- Zoek naar "MAUI App" in de projecttemplates en selecteer deze.
- Klik op "Next".

3. **Configureer je project**:

- Geef je project een naam, bijvoorbeeld "Chapter3".
- Kies een locatie om het project op te slaan.
- Klik op "Create".

4. **Selecteer de standaardtemplate**:

- Selecteer de standaardtemplate voor een nieuwe MAUI-app en klik op "Create".

3.2 Verkennen van de Projectstructuur

Nu je project is aangemaakt, is het tijd om de projectstructuur te verkennen.

3.2 1 Belangrijke onderdelen:

1. **MainPage.xaml**:

- Dit bestand bevat de XAML-definitie van de hoofdgebruikersinterface van je applicatie.

2. **MainPage.xaml.cs**:

- Dit is de code-behind file die de logica voor de gebruikersinterface bevat.

3. **Platforms-map**:

- Deze map bevat platformspecifieke implementaties voor Android en Windows.

4. **App.xaml**:

- Dit bestand definieert globale stijlen en middelen voor je applicatie.

5. **App.xaml.cs**:

- Dit is de code-behind file voor App.xaml, waar de initiële opstartlogica van je applicatie wordt gedefinieerd.

3.3 Aanpassen van de "Hello World" Applicatie

Nu gaan we de standaard "Hello World" applicatie aanpassen.

Stappen:

1. **Open MainPage.xaml**:

- Dubbelklik op MainPage.xaml in de Solution Explorer om het bestand te openen.

2. **Wijzig de gebruikersinterface**:

- Vervang de inhoud van MainPage.xaml met de volgende code:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="HelloMaui.MainPage"
             x:Name="page">

    <VerticalStackLayout>
        <Label
            Text="Welkom bij MAUI!"
            VerticalOptions="Center"
            HorizontalOptions="Center" />

        <Button
            Text="Wissel kleur"
            VerticalOptions="Center"
            HorizontalOptions="Center"
            Clicked="OnButtonClicked"/>
    </VerticalStackLayout>
</ContentPage>
```

3. **Voeg een klik-event toe**:

- Open MainPage.xaml.cs.
- Voeg de volgende methode toe aan de MainPage klasse:

```
private void OnButtonClicked(object sender, EventArgs e)
{
        Random r = new Random();
        page.BackgroundColor =
                Color.FromRgb(r.Next(255), r.Next(255), r.Next(255));
}
```

- Probeer te voorspellen wat de applicatie doet en voer de applicatie uit om te kijken of je het bij het rechte eind hebt.

3.4 Builden en Uitvoeren van de Applicatie

Nu je de gebruikersinterface hebt aangepast, is het tijd om je applicatie te bouwen en uit te voeren op zowel Windows als de Android-emulator.

Stappen:

1. **Selecteer het doelplatform**:

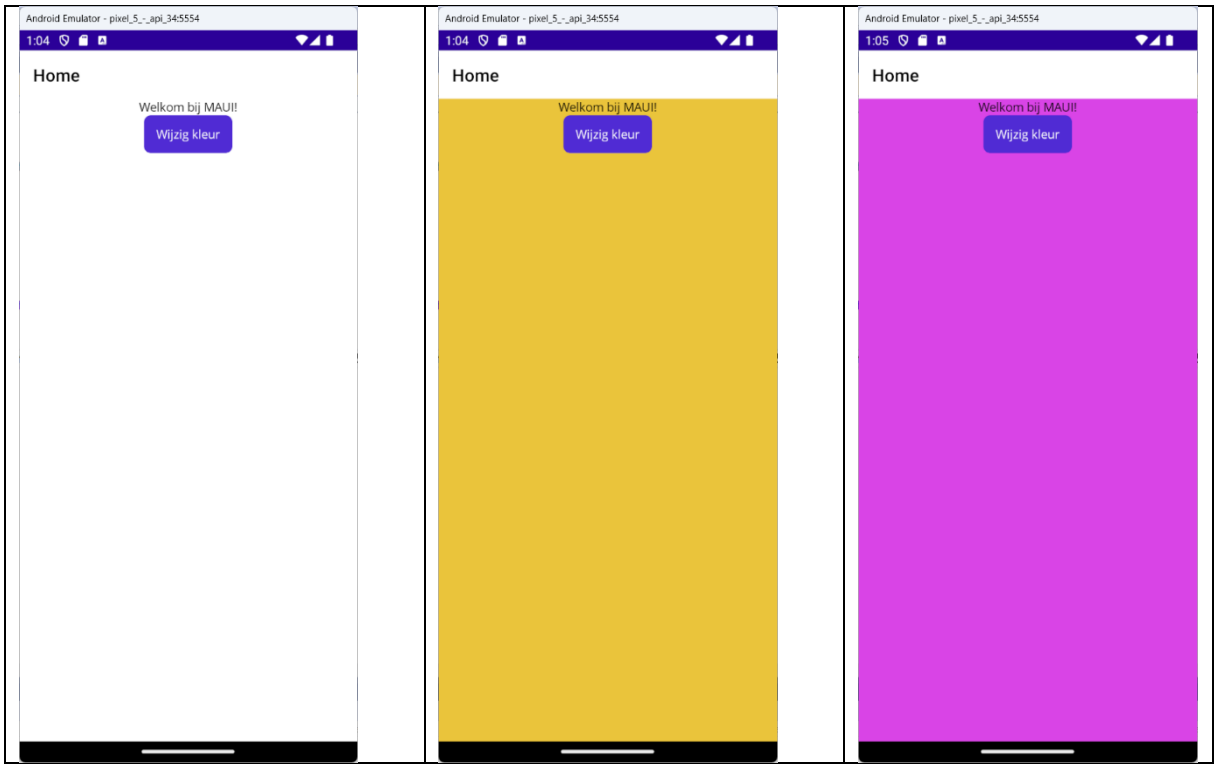
- Bovenaan Visual Studio, selecteer je de target, bijvoorbeeld "Windows Machine" of de naam van je Android-emulator.

2. **Bouw en voer de applicatie uit**:

- Klik op de groene "Start" knop om je applicatie te bouwen en uit te voeren.

3. **Controleer de functionaliteit**:

- Zorg ervoor dat de applicatie correct wordt gestart en dat de kleur van de achtergrond verandert wanneer op de knop wordt geklikt.



Oefeningen

Oefening 7: Maak je Eerste .NET MAUI Project

1. **Doel**: Maak een nieuw .NET MAUI-project aan en verken de projectstructuur.
2. **Stappen**:

- Volg de stappen in sectie 3.1 om een nieuw project aan te maken.
- Verken de verschillende bestanden en mappen in het project.

3. **Controle**: Zorg ervoor dat je de basisstructuur van een .NET MAUI-project begrijpt.

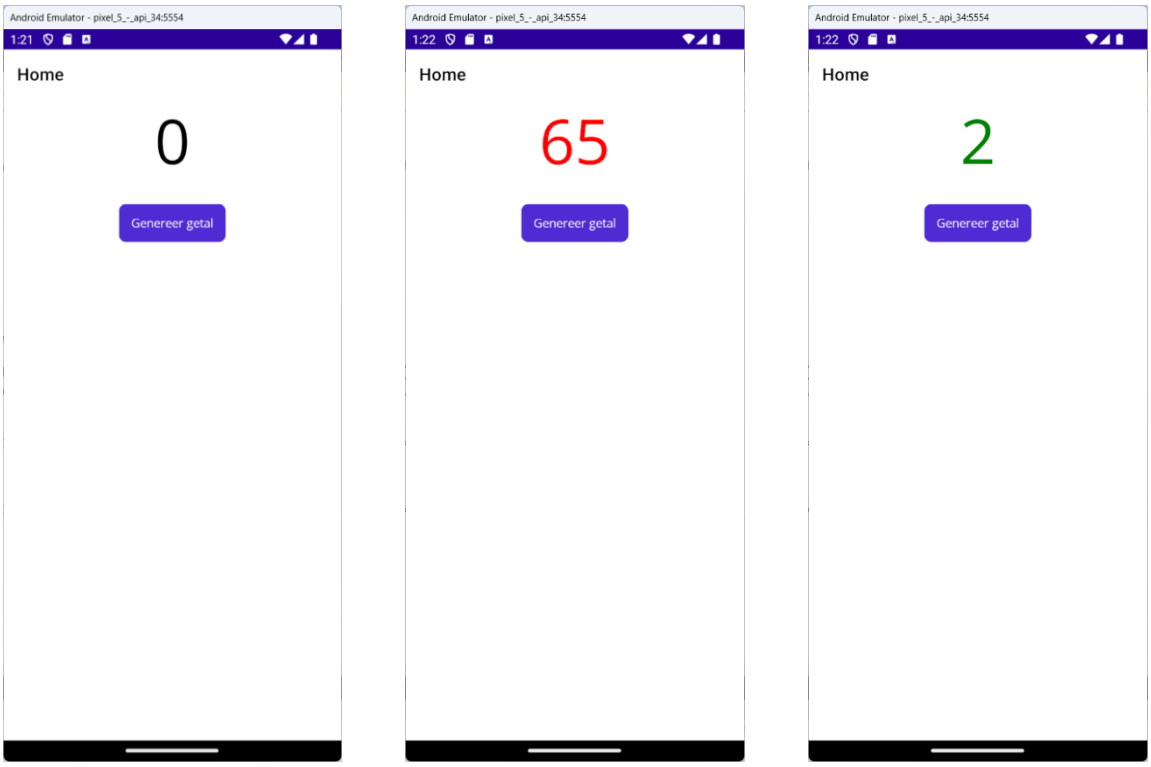
Oefening 8: Pas de "Hello World" Applicatie aan

1. **Doel**: Pas de standaard "Hello World" applicatie aan en zorg dat deze een random getal weergeeft in een groot lettertpe in een kleur die afhankelijk is van de grootte van het getal.
2. **Stappen**:

- Verwijder alle elementen van de pagina.
- Voeg een tekstvak toe aan de pagina. Het vak moet worden gecentreerd op de pagina en de fontsize moet 72 zijn. De startkleur van de tekst is zwart en de startwaarde is 0.
- Voeg onder het tekstvak een knop toe met als tekst Genereer getal.
- Voeg een klik-event toe aan de knop.

- Zorg dat een random getal tussen 1 en 100 wordt weergegeven in het tekstvak als op de knop wordt geklikt. Als het oneven is moet de kleur van het getal rood zijn en als het even is dan moet de kleur groen zijn.

3. **Controle**: Bouw en voer je applicatie uit op zowel Windows als de Android-emulator om te controleren of de wijzigingen correct zijn toegepast.



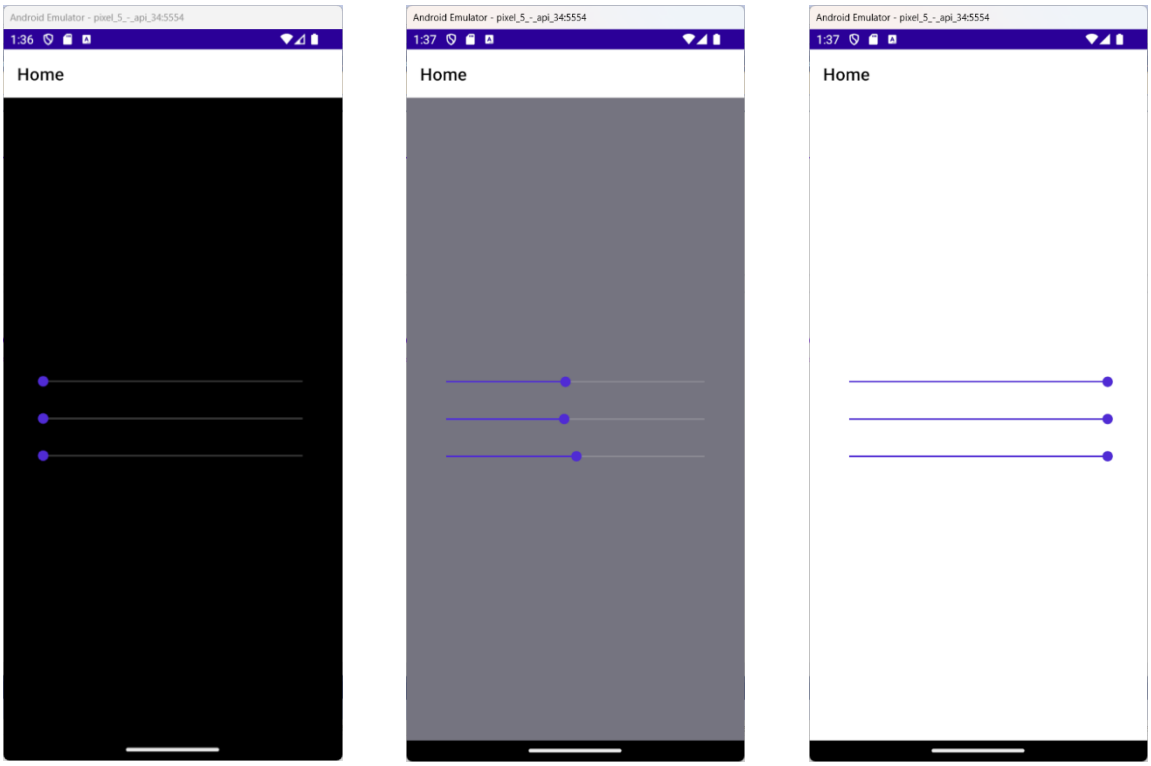
Oefening 9: Experimenteer met XAML

1. **Doel**: Experimenteer met het aanpassen van de gebruikersinterface in XAML.
2. **Stappen**:

- Voeg extra elementen toe aan MainPage.xaml, zoals een Entry voor tekstinvoer of een Slider.
- Speel met de eigenschappen van de elementen, zoals kleuren, marges en paddings.

3. **Controle**: Bouw en voer je applicatie uit om te zien hoe de wijzigingen de gebruikersinterface beïnvloeden.

Je kunt jezelf uitdagen door een app te maken met 3 sliders, waarmee een kleur voor de achtergrond ingesteld kan worden.



Conclusie

In dit hoofdstuk heb je geleerd hoe je een .NET MAUI-project aanmaakt, de projectstructuur verkent, en een eenvoudige "Hello World" applicatie aanpast. Met deze basiskennis ben je klaar om meer geavanceerde functionaliteit toe te voegen en je vaardigheden verder te ontwikkelen in de volgende hoofdstukken. Laten we doorgaan naar het volgende hoofdstuk en dieper ingaan op het toevoegen van functionaliteit aan je applicaties!

Inleveropdracht 1

Maak een applicatie waarmee kinderen leren rekenen met de volgende 3 basis rekenfunctie: plus, min, maal.

De applicatie genereert twee random getallen tussen 0 en 10 (grenzen niet meegerekend, dus 1 tot en met 9) en kiest een random berekening. Vervolgens wordt de berekening inclusief de uitkomst getoond.

Houd verder nog rekening met de volgende eisen:

- Laat AI een toepasselijke afbeelding genereren voor je applicatie.
- Bij een plus-som is de achtergrondkleur van het scherm rood en de kleur van de tekst is geel.
- Bij een min-som is de achtergrondkleur van het scherm blauw en de kleur van de tekst is wit.
- Bij een maal-som is de achtergrondkleur van het scherm geel en de kleur van de tekst is rood.
- De grootte van de tekst is 24.
- De tekst moet horizontaal worden gecentreerd op de pagina.
- De kop van de applicatie heeft als tekst *Leren rekenen*.

Door op een knop te klikken kan een nieuwe som worden gegenereerd.

Het eindresultaat komt er ongeveer zo uit te zien:

Hoofdstuk 4: Werken met layouts

In .NET MAUI zijn verschillende lay-outopties beschikbaar om de gebruikersinterface (UI) van je applicatie te structureren. In dit hoofdstuk bespreken we de belangrijkste lay-outs, waaronder `StackLayout`, `Grid`, `AbsoluteLayout` en `FlexLayout`. Binnen deze layouts kun je gebruik maken van een grote hoeveelheid controls, waarvan we er in dit hoofdstuk een aantal de revue laten passeren.

4.1 StackLayout

`StackLayout` plaatst de kinderen in een enkele rij of kolom, afhankelijk van de `Orientation` eigenschap.

```
<StackLayout Orientation="Vertical" Padding="10">
    <Label Text="First Label" />
    <Label Text="Second Label" />
    <Button Text="Click Me" />
</StackLayout>
```

4.1.1 Horizontal StackLayout:

```
<StackLayout Orientation="Horizontal" Padding="10">
    <Label Text="Label 1" />
    <Label Text="Label 2" />
    <Button Text="Button" />
</StackLayout>
```

4.1.2 Vertical StackLayout:

```
<StackLayout Orientation="Vertical" Padding="10">
    <Label Text="Label 1" />
    <Label Text="Label 2" />
    <Button Text="Button" />
</StackLayout>
```

4.2 Grid

`Grid` verdeelt de ruimte in rijen en kolommen, waardoor je een raster kunt maken voor een complexere lay-out.

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="2*" />
    </Grid.ColumnDefinitions>

    <Label Grid.Row="0" Grid.Column="0" Text="Name:" />
    <Entry Grid.Row="0" Grid.Column="1" />
    <Label Grid.Row="1" Grid.Column="0" Text="Email:" />
    <Entry Grid.Row="1" Grid.Column="1" />
</Grid>
```

4.3 AbsoluteLayout

`AbsoluteLayout` positioneert kinderen op absolute posities, ideaal voor scenario's waar je volledige controle over de positie en grootte van de elementen nodig hebt.

```
<AbsoluteLayout>
    <Label Text="Top Left"
           AbsoluteLayout.LayoutBounds="0, 0, AutoSize, AutoSize"
           AbsoluteLayout.LayoutFlags="PositionProportional" />
    <Label Text="Bottom Right"
           AbsoluteLayout.LayoutBounds="1, 1, AutoSize, AutoSize"
           AbsoluteLayout.LayoutFlags="PositionProportional" />
</AbsoluteLayout>
```

4.4 FlexLayout

`FlexLayout` is een krachtige lay-out die lijkt op CSS Flexbox, waarmee je flexibele en responsieve lay-outs kunt maken.

4.4.1 Horizontal FlexLayout:

```
<FlexLayout Direction="Row" JustifyContent="SpaceBetween"
AlignItems="Center">
    <Label Text="Left" />
    <Label Text="Center" />
    <Label Text="Right" />
</FlexLayout>
```

4.4.2 Vertical FlexLayout:

```
<FlexLayout Direction="Column" JustifyContent="Center"
AlignItems="Stretch">
    <Label Text="Top" />
    <Label Text="Middle" />
    <Label Text="Bottom" />
</FlexLayout>
```

Oefeningen

Oefening 10: Basis StackLayout

Maak een `StackLayout` met drie knoppen onder elkaar. De eerste knop heeft de tekst "Knop 1", de tweede "Knop 2" en de derde "Knop 3".

Oefening 11: Grid Layout

Maak een `Grid` met twee rijen en twee kolommen. Plaats labels in elk van de vier cellen en stel de achtergrondkleur in van elke cel voor een duidelijk onderscheid.

Oefening 12: Absolute Layout

Gebruik een `AbsoluteLayout` om drie knoppen te positioneren: een in de linkerbovenhoek, een in het midden en een in de rechteronderhoek van het scherm.

Oefening 13: Flex Layout

Maak een `FlexLayout` die kinderen horizontaal uitlijnt met gelijke ruimte tussen hen. Voeg drie labels toe met de teksten "Links", "Midden" en "Rechts".

Oefening 14: Relative Layout

Gebruik een `RelativeLayout` om een label in het midden van het scherm te plaatsen en een ander label 20 pixels rechts van het eerste label.

Video tutorials

- Understand DotNet MAUI Layouts | DotNet MAUI Tutorial
- .NET MAUI and Layouts

Hoofdstuk 5: Toevoegen van Functionaliteit aan je .NET MAUI Applicatie

Nu je een basiskennis hebt van het aanmaken en aanpassen van een .NET MAUI-project, is het tijd om meer functionaliteit aan je applicatie toe te voegen. In dit hoofdstuk gaan we dieper in op het werken met verschillende controls, het binden van data, en het hanteren van gebruikersinvoer.

5.1 Werken met Verschillende Controls

.NET MAUI biedt een breed scala aan ingebouwde controls waarmee je interactieve en aantrekkelijke gebruikersinterfaces kunt maken. We zullen enkele van de meest gebruikte controls verkennen.

Voorbeeldcontrols:

1. **Label**: Een eenvoudige tekstweergave.
2. **Entry**: Een invoerveld voor tekst.
3. **Button**: Een knop die een actie uitvoert wanneer erop wordt geklikt.
4. **Slider**: Een schuifregelaar voor het selecteren van een waarde binnen een bereik.
5. **Switch**: Een schakelaar voor het in- of uitschakelen van een waarde.

Stappen:

1. **Open MainPage.xaml**:

- Dubbelklik op `MainPage.xaml` in de Solution Explorer om het bestand te openen.

2. **Voeg verschillende controls toe**:

- Voeg de volgende XAML-code toe om de verschillende controls te gebruiken:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
             xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
             x:Class="HelloMaui.MainPage">


    <VerticalStackLayout>
        <Label
             Text="Welkom bij MAUI!"
             VerticalOptions="Center"
             HorizontalOptions="Center" />

        <Entry
             x:Name="entryField"
             Placeholder="Voer wat tekst in"
             VerticalOptions="Center"
             HorizontalOptions="Center" />


        <Button
             Text="Klik op mij"
             VerticalOptions="Center"
             HorizontalOptions="Center"
             Clicked="OnButtonClicked" />

        <Slider
             x:Name="sliderControl"
             Minimum="0"
             Maximum="100"
             VerticalOptions="Center"
             HorizontalOptions="Center" />

        <Switch
             x:Name="switchControl"
             IsToggled=”True”
             VerticalOptions="Center"
             HorizontalOptions="Center" />

        <Label
             x:Name="sliderValueLabel"
             Text="Slider waarde: 0"
             VerticalOptions="Center"
             HorizontalOptions="Center" />
    </VerticalStackLayout>

</ContentPage>
```

3. **Voeg eventhandlers toe in MainPage.xaml.cs**:

- Voeg de volgende methode toe om de sliderwaarde bij te werken:

```
private void OnSliderValueChanged(object sender, ValueChangedEventArgs e)
{
    sliderValueLabel.Text = $"Slider waarde: {e.NewValue}";
}
```

4. **Bind het Slider ValueChanged-event**:

- Voeg het ValueChanged-event toe aan de Slider in `MainPage.xaml`:

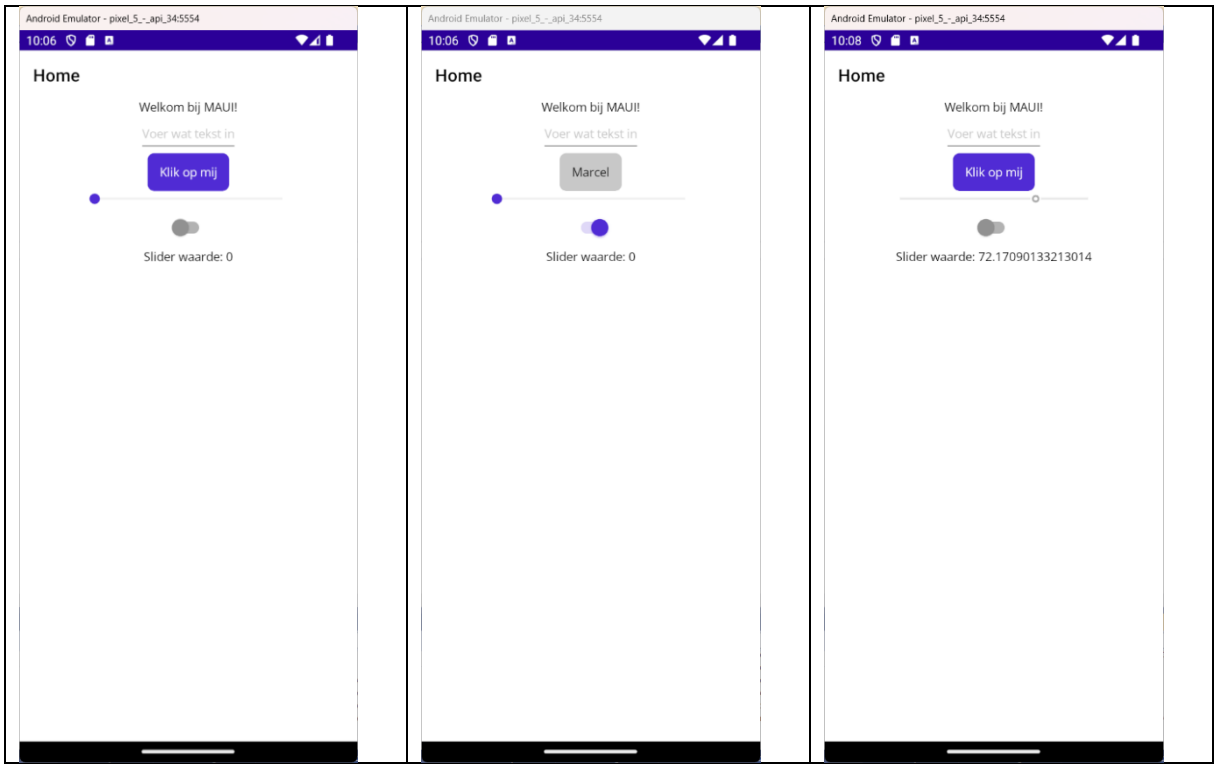
```
<Slider
    x:Name="sliderControl"
    WidthRequest=”250”
    Minimum="0"
    Maximum="100"
    VerticalOptions="Center"
    HorizontalOptions="Center"
    ValueChanged="OnSliderValueChanged" />
```

5. **Voeg een eventhandler toe voor het click event van de button.**

- Zorg dat na het klikken op de button jouw naam verschijnt op de button en dat de button daarna wordt uitgeschakeld.

6. **Voeg een eventhandler toe voor de switch.**

- Zorg dat de slider wordt in- of uitgeschakeld op basis van de waarde van de switch.



5.2 Data Binding

Data binding is een krachtig concept dat je toelaat om de gebruikersinterface en de onderliggende data in je applicatie gesynchroniseerd te houden. MAUI ondersteunt zowel eenvoudige als complexe data binding.

5.2.1 Eenvoudige Data Binding:

1. **Update MainPage.xaml**:

- Voeg een Label en een Entry toe die gebonden zijn aan dezelfde eigenschap:

```
<Label
    Text="{Binding BoundText}"
    VerticalOptions="Center"
    HorizontalOptions="Center" />

<Entry
    Text="{Binding BoundText}"
    VerticalOptions="Center"
    HorizontalOptions="Center" />
```

2. **Maak een ViewModel**:

- Voeg een nieuwe klasse toe genaamd `MainPageViewModel.cs`:

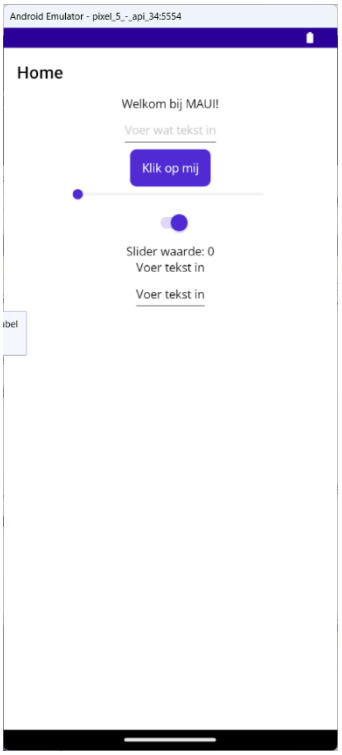
```
public class MainPageViewModel : INotifyPropertyChanged
{
    private string boundText;
    public string BoundText
    {
        get => boundText;
        set
        {
            if (boundText != value)
            {
                boundText = value;
                OnPropertyChanged(nameof(BoundText));
            }
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string propertyName)
    {
             PropertyChanged?.Invoke(this, new
                    PropertyChangedEventArgs(propertyName));
    }
}
```

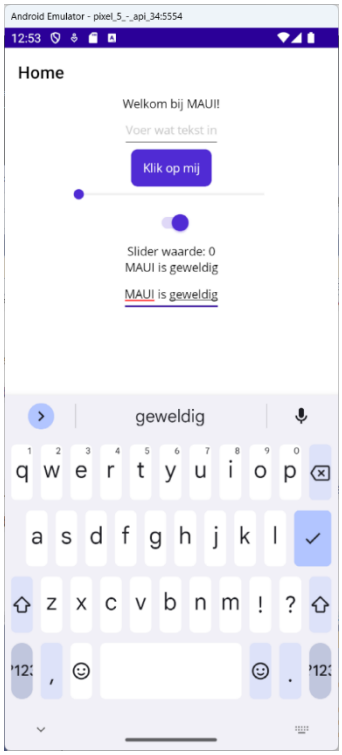
3. **Bind de ViewModel aan de MainPage:**

- Update de `MainPage.xaml.cs` constructor om de BindingContext in te stellen:

```
public MainPage()
{
    InitializeComponent();
    BindingContext = new MainPageViewModel
    {
        BoundText = "Voer tekst in"
    };
}
```

Start de applicatie en wijzig de tekst in het invoervak.





5.3 Hanteren van Gebruikersinvoer

Het hanteren van gebruikersinvoer is een cruciaal aspect van het bouwen van interactieve applicaties. We zullen enkele methoden verkennen om gebruikersinvoer te verwerken.

Voorbeeld:

1. **Klik op een knop en toon invoer:**

- Update de `OnButtonClicked` methode om de tekst van de `entryField` weer te geven:

```
private void OnButtonClicked(object sender, EventArgs e)
{
    string inputText = entryField.Text;
    (sender as Button).Text = $"Invoer: {inputText}";
}
```

Oefeningen

Oefening 15: Voeg Verschillende Controls Toe

1. **Doel**: Voeg verschillende controls toe aan de gebruikersinterface en experimenteer met hun eigenschappen.
2. **Stappen**:
   - Maak eerst een nieuw project aan met als naam **Exercise10**.
   - Verwijder alle controls van de MainPage.xaml pagina.
   - Volg de stappen in sectie 4.1 om verschillende controls toe te voegen.
   - Pas de eigenschappen van de controls aan (bijv. kleuren, marges).
3. **Controle**: Bouw en voer je applicatie uit om te zien hoe de verschillende controls werken.

Oefening 16: Implementeer Eenvoudige Data Binding

1. **Doel**: Implementeer eenvoudige data binding tussen een ViewModel en de gebruikersinterface.
2. **Stappen**:

- Volg de stappen in sectie 4.2 om data binding te implementeren.
- Bind een Label en een Entry aan dezelfde eigenschap in het ViewModel.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de data binding correct werkt.

Oefening 17: Verwerk Gebruikersinvoer

1. **Doel**: Verwerk gebruikersinvoer en toon de resultaten in de gebruikersinterface.
2. **Stappen**:

   - Volg de stappen in sectie 4.3 om gebruikersinvoer te verwerken.
   - Pas de `OnButtonClicked` methode aan om de tekst van de `entryField` weer te geven.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de gebruikersinvoer correct wordt verwerkt en weergegeven.

Video tutorials

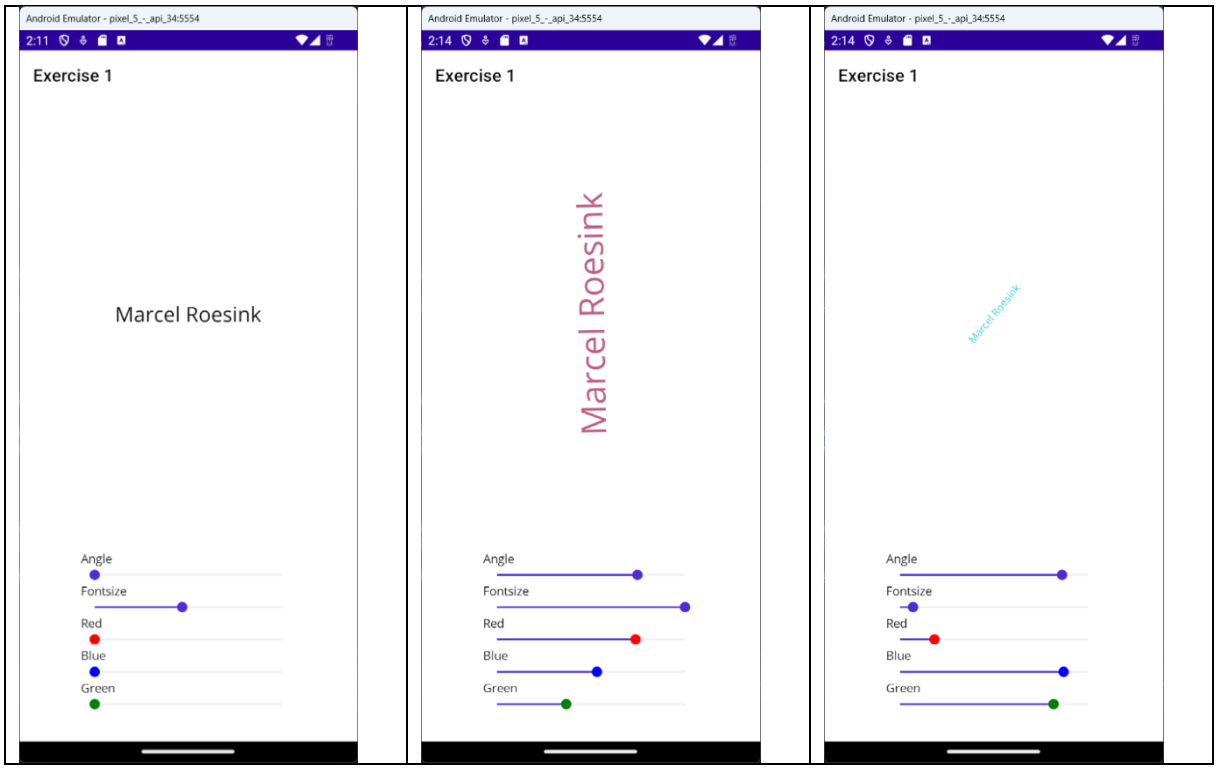
- .NET MAUI Controls: Mastering the Basics and Beyond
- .NET MAUI and Data Bindings

Conclusie

In dit hoofdstuk heb je geleerd hoe je meer functionaliteit kunt toevoegen aan je .NET MAUI-applicatie door verschillende controls te gebruiken, data binding toe te passen, en gebruikersinvoer te verwerken. Deze vaardigheden zijn essentieel voor het bouwen van interactieve en dynamische applicaties. In de volgende hoofdstukken zullen we dieper ingaan op het werken met externe data en het implementeren van geavanceerdere functionaliteiten. Laten we doorgaan en je .NET MAUI-vaardigheden verder ontwikkelen!

Inleveropdracht 2

Maak een applicatie die er op een Pixel5 ongeveer zo uit ziet:



Functionele eisen:

- Met de slider voor *Angle* kan de rotatie van de tekst worden ingesteld op een waarde tussen 0 en 360. De startwaarde is 0. (M)
- Met de slider voor *Fontsize* kan de grootte van de tekst worden ingesteld op een waarde tussen 10 en 40. De startwaarde is 24. (M)
- Met de sliders voor *Red, Blue, Green* kan de kleur van de tekst worden ingesteld. De startwaarde is voor al deze sliders 0. (M)
- De titel van de applicatie is **Exercise 1**. (M)
- De tekst moet natuurlijk jouw eigen naam zijn. (M)

Niet-functionele eisen:

- Als het scherm draait is nog steeds de hele app te zien (S)

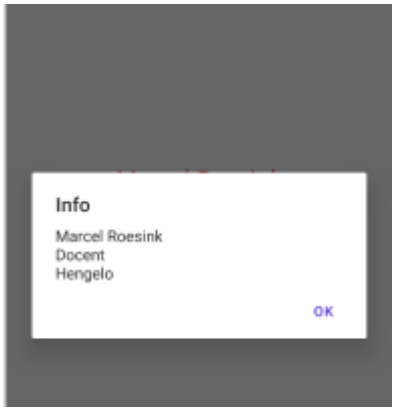
Laat de opdracht werkend zien aan je docent. Zorg dat je uit kunt leggen hoe je eigen code werkt. Maak diverse screenshots en plak die in een Word document. Lever het Word document in Teams in onder opdracht **Inleveropdracht 1**.

Inleveropdracht 3

Maak een app die informatie toont over jouw eerste stageperiode.

Functionele eisen:

- Als kop boven de pagina moet komen **Stage van xxx bij XXX.** Bijvoorbeeld: *Stage van Jaap de Groot bij Nubix*.
- De achtergrondkleur van het scherm moet een kleur zijn uit het logo van jouw stagebedrijf.
- Zorg dat een ander lettertype (font) wordt gebruikt dan het standaard lettertype
- In het scherm moet de volgende informatie worden weergegeven:
    - Een afbeelding/logo van jouw stagebedrijf.
    - De naam van jouw stagebedrijf: fontsize 36, fontweight bold
    - De adresgegevens van het stagebedrijf: fontsize 24 fontstyle italic
        - Straat
        - Huisnummer
        - Postcode en plaats
        - Website als hyperlink
    - Een aantal buttons naast elkaar met de verschillende technieken en talen waarmee jij hebt gewerkt (minimaal 3). De buttons moeten werken als hyperlink naar een webpagina met uitleg over die techniek, bijvoorbeeld naar een subpagina op W3Schools.
    - Een scrollvenster met jouw persoonlijke ervaringen tijdens de stage.
    - Een button die van jou de volgende gegevens toont in een Popup venster:
        - Volledige naam
        - Klas
        - Woonplaats



Laat de opdracht werkend zien aan je docent. Zorg dat je uit kunt leggen hoe je eigen code werkt. Maak diverse screenshots en plak die in een Word document. Lever het Word document in Teams in onder opdracht **Inleveropdracht 2**.

Inleveropdracht 4

Maak een eenvoudige rekenmachine.

Functionele eisen:

1. De gebruiker kan 2 getallen invoeren.
2. De gebruiker kan middels een vijftal knoppen de volgende berekeningen uitvoeren:
    a. Som (getal 1 + getal 2)
    b. Verschil (getal1 – getal 2)
    c. Product (getal 1 * getal 2)
    d. Quotiënt (getal 1 / getal 2)
    e. Restant (getal 1 % getal 2)
3. De uitkomst wordt getoond in 2 decimalen.
4. Bij het berekenen van quotiënt en restant mag getal 2 niet gelijk zijn aan 0. Als dat wel zo is moet met een popup een foutmelding worden getoond om de gebruiker erop te wijzen dat delen door 0 flauwekul is.

Laat de opdracht werkend zien aan je docent. Zorg dat je uit kunt leggen hoe je eigen code werkt. Lever daarna een screenshot van je werkende app in Teams in bij **Inleveropdracht 3**.

Hoofdstuk 6: Geavanceerde Functionaliteiten en Gebruikerservaring

Nu je bekend bent met de basis van .NET MAUI en hebt geleerd hoe je externe data kunt integreren, is het tijd om geavanceerdere functionaliteiten toe te voegen en de gebruikerservaring te verbeteren. In dit hoofdstuk behandelen we navigatie tussen pagina's, het gebruik van ingebouwde functionaliteiten van het apparaat, en het implementeren van animaties en visuele effecten.

6.1 Navigatie tussen Pagina's

Navigatie is een belangrijk onderdeel van elke mobiele applicatie. .NET MAUI biedt eenvoudige manieren om tussen pagina's te navigeren.

Voorbeeld:

1. **Aanmaken van een tweede pagina**:
    - Voeg een nieuwe ContentPage toe aan je project genaamd `SecondPage.xaml`:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
             xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
             x:Class="HelloMaui.SecondPage">

    <StackLayout>
        <Label Text="Dit is de tweede pagina!"
               VerticalOptions="CenterAndExpand"
               HorizontalOptions="CenterAndExpand" />
    </StackLayout>

</ContentPage>
```

2. **Navigeren naar de tweede pagina**:
    - Update de `MainPage.xaml` om een knop toe te voegen die naar de tweede pagina navigeert:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
             xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
             x:Class="HelloMaui.MainPage">

    <StackLayout>
        <Button Text="Ga naar de tweede pagina"
                Clicked="OnNavigateButtonClicked" />
    </StackLayout>

</ContentPage>
```

3. **Implementeren van de navigatielogica**:

- Voeg de volgende methode toe aan `MainPage.xaml.cs`:

```
private async void OnNavigateButtonClicked(object sender, EventArgs e)
{
    await Navigation.PushAsync(new SecondPage());
}
```

4. **Configureer navigatie in de App klasse**:

- Update App.xaml.cs om een NavigationPage te gebruiken:

```
public App()
{
    InitializeComponent();

    MainPage = new NavigationPage(new MainPage());
}
```

6.2 Gebruik van Ingebouwde Functionaliteiten van het Apparaat

.NET MAUI biedt toegang tot verschillende ingebouwde functionaliteiten van het apparaat, zoals de camera, geolocatie, en sensoren.

Voorbeeld: Toegang tot de Camera

1. **Toevoegen van de benodigde NuGet-pakketten**:

- Voeg het Xamarin.Essentials NuGet-pakket toe aan je project.

2. **Vraag om toestemming**:

- Zorg ervoor dat je de juiste toestemmingen hebt toegevoegd aan je AndroidManifest.xml en Info.plist voor respectievelijk Android en iOS.

3. **Implementeren van de cameralogica**:

- Voeg een knop toe aan MainPage.xaml om een foto te maken:

```
<Button Text="Maak een foto"
        Clicked="OnCameraButtonClicked" />
```

- Voeg de volgende methode toe aan MainPage.xaml.cs:

```
private async void OnCameraButtonClicked(object sender, EventArgs e)
{
    var photo = await MediaPicker.CapturePhotoAsync();
    if (photo != null)
    {
        var stream = await photo.OpenReadAsync();
        // Verwerk de foto (bijv. toon deze in een Image control)
    }
}
```

6.3 Implementeren van Animaties en Visuele Effecten

Animaties en visuele effecten kunnen je applicatie aantrekkelijker en gebruiksvriendelijker maken. .NET MAUI biedt verschillende ingebouwde animaties.

Voorbeeld: Een eenvoudige animatie

1. **Voeg een knop en een label toe aan `MainPage.xaml`:**

```
<StackLayout>
    <Button Text="Animate" Clicked="OnAnimateButtonClicked" />
    <Label x:Name="AnimatedLabel" Text="Hello, MAUI!"
           VerticalOptions="CenterAndExpand"
           HorizontalOptions="CenterAndExpand" />
</StackLayout>
```

2. **Implementeer de animatielogica in `MainPage.xaml.cs`:**

```
private async void OnAnimateButtonClicked(object sender, EventArgs e)
{
    await AnimatedLabel.TranslateTo(100, 0, 1000, Easing.BounceOut);
    await AnimatedLabel.TranslateTo(0, 0, 1000, Easing.BounceIn);
}
```

Oefeningen

Oefening 18: Navigeren tussen Pagina's

1. **Doel**: Implementeer navigatie tussen verschillende pagina's in je applicatie.
2. **Stappen**:

- Volg de stappen in sectie 6.1 om een tweede pagina aan te maken en navigatielogica toe te voegen.
- Voeg extra informatie toe aan de tweede pagina en navigeer heen en weer.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de navigatie correct werkt.

Oefening 19: Gebruik van de Camera

1. **Doel**: Voeg functionaliteit toe aan je applicatie om foto's te maken met de camera.
2. **Stappen**:

- Volg de stappen in sectie 6.2 om toegang te krijgen tot de camera en een foto te maken.
- Toon de gemaakte foto in de gebruikersinterface.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de camerafunctionaliteit correct werkt.

Oefening 20: Implementeer een Animatie

1. **Doel**: Voeg een eenvoudige animatie toe aan je applicatie.
2. **Stappen**:

- Volg de stappen in sectie 6.3 om een animatie te implementeren.
- Experimenteer met verschillende animaties en effecten.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de animaties correct werken.

Conclusie

In dit hoofdstuk heb je geleerd hoe je geavanceerde functionaliteiten kunt toevoegen en de gebruikerservaring kunt verbeteren in je .NET MAUI-applicaties. Je hebt gewerkt met navigatie, toegang tot apparaat functies, en het implementeren van animaties. Met deze kennis kun je rijkere en meer interactieve applicaties bouwen. In de volgende hoofdstukken zullen we ons richten op het optimaliseren van prestaties en het implementeren van best practices. Laten we doorgaan en je .NET MAUI-vaardigheden verder ontwikkelen!

Inleveropdracht 5

Doel van de Opdracht

Het doel van deze opdracht is om je vertrouwd te maken met het gebruik van geavanceerde functionaliteiten binnen .NET MAUI, waaronder navigatie tussen pagina's, het gebruik van ingebouwde apparaat functionaliteiten en het implementeren van animaties en visuele effecten. Je zult een eenvoudige applicatie bouwen die deze elementen combineert om een vloeiende en interactieve gebruikerservaring te creëren.

Opdrachtomschrijving

Je gaat een mobiele applicatie bouwen met .NET MAUI die de volgende functies bevat:

1. **Navigatie tussen Pagina's**:
    - Maak ten minste drie verschillende pagina's in je applicatie: een hoofdpagina, een instellingenpagina, en een informatiepagina.
    - Implementeer navigatie tussen deze pagina's met behulp van de NavigationPage.
    - Voeg een knop toe op de hoofdpagina die de gebruiker naar de instellingenpagina brengt, en een andere knop die naar de informatiepagina navigeert.
2. **Gebruik van Ingebouwde Functionaliteiten van het Apparaat**:
    - Implementeer op de instellingenpagina een functionaliteit die de gebruiker toestaat de huidige locatie (GPS) op te halen en weer te geven op de pagina.
    - Voeg een knop toe waarmee de gebruiker de camera kan openen om een foto te maken. Toon de gemaakte foto vervolgens op de informatiepagina.
3. **Implementeren van Animaties en Visuele Effecten**:
    - Voeg een eenvoudige animatie toe op de hoofdpagina, bijvoorbeeld een fade-in effect wanneer de pagina wordt geladen of een knop die pulseert wanneer erop geklikt wordt.
    - Implementeer een visueel effect op de navigatie tussen pagina's, zoals een overgangsanimatie (bijvoorbeeld een slide- of fade-effect) wanneer de gebruiker van de ene naar de andere pagina navigeert.

Stappenplan

1. **Stap 1: Opzetten van de Applicatie**
   Begin met het aanmaken van een nieuw .NET MAUI-project in Visual Studio. Voeg de verschillende pagina's (MainPage, SettingsPage, InfoPage) toe aan het project.
2. **Stap 2: Implementatie van Navigatie**
    - Configureer de NavigationPage en implementeer de navigatie tussen de pagina's. Zorg ervoor dat de knoppen correct zijn gekoppeld aan de juiste navigatiemethodes.
3. **Stap 3: Ingebouwde Functionaliteiten van het Apparaat**

- Voeg code toe om de huidige GPS-locatie op te halen en weer te geven op de instellingenpagina.
- Implementeer de camerafunctionaliteit zodat een foto kan worden genomen en weergegeven op de informatiepagina.

4. **Stap 4: Toevoegen van Animaties en Visuele Effecten**
    - Gebruik de animatie- en transitiefunctionaliteiten van .NET MAUI om visuele effecten toe te voegen aan je applicatie. Experimenteer met verschillende animaties om een vloeiende gebruikerservaring te creëren.

Hoofdstuk 7: Verschillende Soorten Pagina's in .NET MAUI

In dit hoofdstuk richten we ons op de verschillende soorten pagina's die je kunt gebruiken in een .NET MAUI-applicatie. .NET MAUI biedt verschillende paginatypen die je kunt gebruiken om je applicatie te structureren en te navigeren. We zullen de meest gebruikte paginatypen bespreken: `ContentPage`, `FlyoutPage`, `TabbedPage`, en `NavigationPage`.

7.1 ContentPage

`ContentPage` is de meest basale en veelgebruikte paginatypen in .NET MAUI. Het wordt gebruikt voor een enkelvoudige pagina met inhoud.

Voorbeeld: Een Eenvoudige ContentPage

```
public partial class HomePage : ContentPage
{
    public HomePage()
    {
        InitializeComponent();
        Content = new StackLayout
        {
            Children = {
                new Label { Text = "Welcome to .NET MAUI!" }
            }
        };
    }
}
```

Gebruik in XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="HelloMaui.HomePage">
    <StackLayout>
        <Label Text="Welcome to .NET MAUI!"
               VerticalOptions="CenterAndExpand"
               HorizontalOptions="CenterAndExpand" />
    </StackLayout>
</ContentPage>
```

7.2 FlyoutPage

`FlyoutPage` (voorheen bekend als MasterDetailPage) is een paginatypen dat een navigatielade (flyout) biedt. Het wordt meestal gebruikt voor navigatie in een applicatie met meerdere secties.

Voorbeeld: Een Eenvoudige FlyoutPage

1. **Definieer de FlyoutPage**

```
public partial class MainFlyoutPage : FlyoutPage
{
    public MainFlyoutPage()
    {
        Flyout = new MenuPage();
        Detail = new NavigationPage(new HomePage());
    }
}
```

2. **Definieer de MenuPage**

```
public partial class MenuPage : ContentPage
{
    public MenuPage()
    {
        InitializeComponent();
        Title = "Menu";
        Content = new StackLayout
        {
            Children = {
                new Button
                {
                    Text = "Home",
                    Command = new Command(() =>
             ((FlyoutPage)Application.Current.MainPage).Detail = new
             NavigationPage(new HomePage()))
                },
                new Button
                {
                    Text = "Settings",
                    Command = new Command(() =>
             ((FlyoutPage)Application.Current.MainPage).Detail = new
             NavigationPage(new SettingsPage()))
                }
            }
        };
    }
}
```

3. **Gebruik in XAML**

```
<?xml version="1.0" encoding="utf-8" ?>
<FlyoutPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
            xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
            x:Class="HelloMaui.MainFlyoutPage">
    <FlyoutPage.Flyout>
        <ContentPage Title="Menu">
            <StackLayout>
                <Button Text="Home" Clicked="OnHomeButtonClicked" />
                <Button Text="Settings" Clicked="OnSettingsButtonClicked"
>
            </StackLayout>
        </ContentPage>
    </FlyoutPage.Flyout>
    <FlyoutPage.Detail>
        <NavigationPage>
            <x:Arguments>
                <local:HomePage />
            </x:Arguments>
        </NavigationPage>
    </FlyoutPage.Detail>
</FlyoutPage>
```

7.3 TabbedPage

`TabbedPage` is een paginatypen dat tabbladen biedt voor het navigeren tussen verschillende pagina's. Dit type pagina wordt vaak gebruikt voor het organiseren van inhoud in secties.

Voorbeeld: Een Eenvoudige TabbedPage

```
public partial class MainTabbedPage : TabbedPage
{
    public MainTabbedPage()
    {
        Children.Add(new HomePage { Title = "Home" });
        Children.Add(new SettingsPage { Title = "Settings" });
    }
}
```

Gebruik in XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<TabbedPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
            xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
            x:Class="HelloMaui.MainTabbedPage">
    <TabbedPage.Children>
        <local:HomePage Title="Home" />
        <local:SettingsPage Title="Settings" />
    </TabbedPage.Children>
</TabbedPage>
```

7.4 NavigationPage

`NavigationPage` is een paginatypen dat navigatie met een stack biedt, waardoor je door verschillende pagina's kunt navigeren met een terugknop.

7.4.1 Voorbeeld: Een Eenvoudige NavigationPage

1. **Definieer de NavigationPage**

```
public partial class App : Application
{
    public App()
    {
        MainPage = new NavigationPage(new HomePage());
    }
}
```

2. **Navigeren tussen Pagina's**

```
public partial class HomePage : ContentPage
{
    public HomePage()
    {
        InitializeComponent();
        Content = new StackLayout
        {
            Children = {
                new Button
                {
                    Text = "Go to Settings",
                    Command = new Command(async () => await
                          Navigation.PushAsync(new SettingsPage()))

                }
            }
        };
    }
}
```

3. **Gebruik in XAML**

```
<?xml version="1.0" encoding="utf-8" ?>
<NavigationPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
                xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
                x:Class="HelloMaui.AppShell">
    <x:Arguments>
        <local:HomePage />
    </x:Arguments>
</NavigationPage>
```

Oefeningen

Oefening 21: Creëer een ContentPage

1. **Doel**: Maak een eenvoudige `ContentPage` met een label en een knop.
2. **Stappen**:

- Definieer een nieuwe ContentPage.
- Voeg een label en een knop toe in de XAML.
- Voeg een Clicked-event toe aan de knop die de tekst van het label verandert.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de `ContentPage` correct werkt.

Oefening 22: Implementeer een FlyoutPage

1. **Doel**: Maak een `FlyoutPage` met een menu en een detailpagina.
2. **Stappen**:
    - Definieer een nieuwe `FlyoutPage`.

- Voeg een menu met knoppen toe in de flyout.
- Voeg verschillende pagina's toe in het detailgedeelte.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de `FlyoutPage` correct werkt.

Oefening 23: Implementeer een TabbedPage

1. **Doel**: Maak een `TabbedPage` met twee tabbladen.
2. **Stappen**:

- Definieer een nieuwe TabbedPage.
- Voeg twee verschillende pagina's toe als tabbladen.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de `TabbedPage` correct werkt.

Oefening 24: Implementeer een NavigationPage

1. **Doel**: Maak een `NavigationPage` en implementeer navigatie tussen twee pagina's.
2. **Stappen**:

- Definieer een nieuwe NavigationPage.
- Voeg een knop toe aan de startpagina die navigeert naar een tweede pagina.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de `NavigationPage` correct werkt.

Conclusie

In dit hoofdstuk heb je geleerd over de verschillende soorten pagina's die beschikbaar zijn in .NET MAUI: `ContentPage`, `FlyoutPage`, `TabbedPage`, en `NavigationPage`. Elk van deze paginatypen heeft zijn eigen specifieke gebruiksscenario's en helpt je bij het organiseren en navigeren binnen je applicatie. In de volgende hoofdstukken zullen we verder gaan met het verkennen van geavanceerde navigatiepatronen en het implementeren van pushnotificaties. Laten we doorgaan en je .NET MAUI-vaardigheden verder ontwikkelen!

Inleveropdracht 6

Opdrachtbeschrijving:

Ontwikkel een mobiele applicatie met .NET MAUI waarin je verschillende soorten pagina's implementeert. De app moet gebruik maken van diverse paginatypes zoals ContentPage, FlyoutPage, TabbedPage, en NavigationPage, elk met een specifieke rol binnen de app.

Scenario:

Je ontwikkelt een reisplanner-app waarin gebruikers hun reis kunnen plannen, beheren en navigeren tussen verschillende secties van de app. De app moet gebruik maken van verschillende paginatypes om een soepele en logische navigatie-ervaring te bieden.

Eisen:

1. **ContentPage:**
    - Implementeer een eenvoudige ContentPage die fungeert als het startscherm van de app. Dit scherm moet gebruikers verwelkomen en basisinformatie geven over de app, zoals de naam van de app en enkele acties (bijvoorbeeld "Reis Plannen" of "Bekijk Mijn Reizen").
    - Gebruik XAML voor de lay-out van deze pagina.
2. **FlyoutPage:**
    - Gebruik een FlyoutPage om een navigatiemenu te maken waarmee gebruikers kunnen wisselen tussen verschillende secties van de app, zoals "Mijn Reizen", "Reis Plannen", en "Instellingen".
    - Zorg ervoor dat de FlyoutPage op een intuïtieve manier is georganiseerd, met duidelijke en logische navigatieopties.
3. **TabbedPage:**
    - Implementeer een TabbedPage om verschillende gerelateerde functies op een logische manier te groeperen. Bijvoorbeeld, de "Mijn Reizen" sectie kan tabs hebben voor "Aankomende Reizen", "Afgeronde Reizen", en "Favoriete Reizen".
    - Elk tabblad moet een eigen ContentPage bevatten die de bijbehorende informatie toont.
4. **NavigationPage:**
    - Gebruik een NavigationPage om diepere navigatie binnen de app mogelijk te maken. Bijvoorbeeld, wanneer een gebruiker een specifieke reis selecteert uit de lijst, kan hij/zij door navigeren naar een gedetailleerde weergave van die reis.
    - Zorg ervoor dat gebruikers gemakkelijk kunnen terugnavigeren naar de vorige pagina's.

Projectvereisten:

- De reisplanner-app moet minstens vier hoofdschermen hebben: een startscherm (ContentPage), een sectie voor het beheren van reizen (TabbedPage), een menu voor navigatie (FlyoutPage), en een detailscherm voor specifieke reizen (NavigationPage).
- De verschillende paginatypes moeten op een logische en gebruikersvriendelijke manier worden gecombineerd om een intuïtieve navigatie-ervaring te bieden.

Beoordelingscriteria:

- **Gebruik van Verschillende Paginatypes:** Zijn alle vereiste paginatypes correct geïmplementeerd en worden ze effectief gebruikt in de app?
- **Navigatie:** Is de navigatie tussen de verschillende pagina's soepel en intuïtief? Functioneren de FlyoutPage, TabbedPage, en NavigationPage zoals verwacht?
- **Gebruikerservaring:** Is de interface duidelijk, aantrekkelijk en gemakkelijk te gebruiken? Zijn de verschillende secties van de app logisch georganiseerd?
- **Codekwaliteit:** Is de code schoon, goed georganiseerd, en zijn de verschillende paginatypes op een logische manier geïntegreerd?

Op te leveren:

1. De broncode van de applicatie, inclusief de implementatie van alle paginatypes.
2. Een korte beschrijving van de structuur van de app en hoe je de verschillende paginatypes hebt gecombineerd om de navigatie te realiseren.
3. Screenshots of een korte video waarin je de belangrijkste functies en navigatie van de app demonstreert.

Hoofdstuk 8: Werken met externe data en API's

In dit hoofdstuk gaan we verkennen hoe je externe data kunt ophalen en gebruiken in je .NET MAUI-applicaties. Dit omvat het werken met web-API's om data op te halen en weer te geven in je applicatie.

8.1 Het Ophalen van data van een Web-API

Het ophalen van data van een web-API is een veelvoorkomende taak in moderne applicaties. We zullen een eenvoudige applicatie maken die data ophaalt van een publieke API en deze weergeeft in de gebruikersinterface.

We maken voor dit hoofdstuk een nieuw project aan met als naam **Chapter5**.

Voorbeeld:

1. **Maak een nieuwe klasse voor de data**:

- Voeg een nieuwe klasse toe genaamd Post.cs:

```
public class Post
{
    public int UserId { get; set; }
    public int Id { get; set; }
    public string Title { get; set; }
    public string Body { get; set; }
}
```

2. **Maak een serviceklasse voor het ophalen van data**:

- Voeg een nieuwe klasse toe genaamd DataService.cs:

```
public class DataService
{
    private readonly HttpClient _httpClient;

    public DataService()
    {
        _httpClient = new HttpClient();
    }

    public async Task<List<Post>> GetPostsAsync()
    {
        var response = await
_httpClient.GetStringAsync("https://jsonplaceholder.typicode.com/posts");
        return JsonConvert.DeserializeObject<List<Post>>(response);
    }
}
```

3. **Update de MainPageViewModel om de data op te halen**:

- Maak een viewmodel MainPageViewModel.cs aan:

```
public class MainPageViewModel : INotifyPropertyChanged
{
    private List<Post> _posts;
    public List<Post> Posts
    {
        get => _posts;
        set
        {
            if (_posts != value)
            {
                _posts = value;
                OnPropertyChanged(nameof(Posts));
            }
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }

    private readonly DataService _dataService;

    public MainPageViewModel()
    {
        _dataService = new DataService();
        LoadData();
    }

    private async void LoadData()
    {
        Posts = await _dataService.GetPostsAsync();
    }
}
```

4. **Bind de data aan de gebruikersinterface**:

- Update de MainPage.xaml om de data weer te geven:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
             xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
             x:Class="HelloMaui.MainPage">

    <CollectionView ItemsSource="{Binding Posts}">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <StackLayout Padding="10">
                    <Label Text="{Binding Title}" FontAttributes="Bold"/>
                    <Label Text="{Binding Body}" />
                </StackLayout>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>

</ContentPage>
```

5. **Verbind de ViewModel met de View**:

- Zorg ervoor dat de BindingContext in de MainPage is ingesteld zoals eerder beschreven.

8.2 Werken met JSON Data

Veel web-API's leveren data in JSON-formaat. Het is belangrijk om te weten hoe je deze JSON-data kunt deserialiseren naar C# objecten.

Stappen:

1. **Installeren van NewtonSoft.Json**:

- Voeg het NuGet-pakket Newtonsoft.Json toe aan je project via de NuGet Package Manager.

2. **Deserialiseren van JSON**:

- Gebruik de JsonConvert klasse om JSON-data te deserialiseren, zoals getoond in de DataService klasse hierboven.

8.3 Afhandelen van Fouten en Netwerkproblemen

Het is belangrijk om fouten en netwerkproblemen op te vangen en gebruikers feedback te geven wanneer iets misgaat.

Voorbeeld:

1. **Update de `DataService` om fouten te handelen:**

```
public class DataService
{
    private readonly HttpClient _httpClient;

    public DataService()
    {
        _httpClient = new HttpClient();
    }

    public async Task<List<Post>> GetPostsAsync()
    {
        try
        {
            var response = await
_httpClient.GetStringAsync("https://jsonplaceholder.typicode.com/posts");
            return JsonConvert.DeserializeObject<List<Post>>(response)!;
        }
        catch (Exception ex)
        {
            // Log de fout (bijv. naar een logging service) en geef een
                    gebruikersvriendelijke boodschap
            Debug.WriteLine($"Fout bij het ophalen van data:
             {ex.Message}");
            return new List<Post>(); // Leeg lijst als fallback
        }
    }
}
```

2. **Geef feedback aan de gebruiker:**

- Voeg een Label toe aan MainPage.xaml om fouten weer te geven:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
             xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
             x:Class="HelloMaui.MainPage">

    <StackLayout>
        <Label x:Name="ErrorLabel" TextColor="Red" IsVisible="False" />
        <CollectionView ItemsSource="{Binding Posts}">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <StackLayout Padding="10">
                        <Label Text="{Binding Title}"
                          FontAttributes="Bold" />
                        <Label Text="{Binding Body}" />
                    </StackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </StackLayout>

</ContentPage>
```

- Update MainPageViewModel om de foutboodschap in te stellen:

```
private async void LoadData()
{
    try
    {
        Posts = await _dataService.GetPostsAsync();
    }
    catch (Exception ex)
    {
        // Geef een foutboodschap weer
        (App.Current.MainPage as MainPage)?.
             ShowError("Kon de data niet laden. Probeer het later
             opnieuw.");
    }
}
```

- Voeg een methode toe in MainPage.xaml.cs om de foutboodschap weer te geven:

```
public void ShowError(string message)
{
    ErrorLabel.Text = message;
    ErrorLabel.IsVisible = true;
}
```

Oefeningen

Oefening 25: Haal Data op van een Web-API

1. **Doel**: Haal data op van een web-API en geef deze weer in de gebruikersinterface.

2. **Stappen**:
    - Volg de stappen in sectie 5.1 om data van een API op te halen en weer te geven.
    - Gebruik een publieke API naar keuze, bijvoorbeeld https://thedogapi.com of https://thecatapi.com/ of https://sv443.net/jokeapi/v2/
    - Zorg dat de lijst scrollable is
3. **Controle**: Bouw en voer je applicatie uit om te controleren of de data correct wordt opgehaald en weergegeven.

Oefening 26: Verwerk JSON Data

1. **Doel**: Deserialiseer JSON data naar C# objecten.
2. **Stappen**:

- Voeg het Newtonsoft.Json NuGet-pakket toe aan je project.
- Deserialiseer JSON data naar een lijst van objecten zoals getoond in sectie 5.2.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de JSON data correct wordt verwerkt.

Oefening 27: Handel Fouten en Netwerkproblemen

1. **Doel**: Implementeer foutafhandeling en geef gebruikers feedback bij netwerkproblemen.
2. **Stappen**:

- Volg de stappen in sectie 5.3 om fouten en netwerkproblemen af te handelen.
- Voeg een Label toe aan de gebruikersinterface om foutboodschappen weer te geven.

3. **Controle**: Bouw en voer je applicatie uit en simuleer netwerkproblemen om te controleren of de fouten correct worden afgehandeld en weergegeven.

Conclusie

In dit hoofdstuk heb je geleerd hoe je externe data kunt ophalen en gebruiken in je .NET MAUI-applicaties. Je hebt gewerkt met web-API's, JSON data, en het handelen van fouten en netwerkproblemen. Met deze kennis kun je nu interactieve applicaties bouwen die data van externe bronnen gebruiken. In de volgende hoofdstukken zullen we dieper ingaan op het toevoegen van meer geavanceerde functionaliteiten en het verbeteren van de gebruikerservaring. Laten we doorgaan en je vaardigheden verder ontwikkelen!

Inleveropdracht 7

De rijksdienst voor het wegverkeer (RDW) stelt een openbare API beschikbaar, waarmee heel veel data geraadpleegd kan worden met betrekking tot alles wat met verkeer te maken heeft. Bijvoorbeeld:

- Basisregistratie kentekens
- Parkeergebieden in Nederland
- Dynamische parkeergegevens
- Bedrijven met RDW-erkenning en bevoegdheid
- Etc.

Voor deze opdracht moet je een app maken die een call doet naar een API van RDW.

Keuzeopdracht 7.1 – maximaal 30 van 50 punten

Met een call naar de API wordt alle informatie opgehaald van parkeerlocaties in Hengelo. Alleen de feitelijke adressen zijn van belang Je hebt de volgende url nodig:

https://opendata.rdw.nl/resource/ygq4-hh5q.json

Documentatie bij de API vind je hier.

De app moet in het scherm de volgende informatie tonen van de gevonden parkeerlocaties:

- Adres (straatnaam + huisnummer)
- Postcode
- E-mailadres

Keuzeopdracht 7.2 – maximaal 30 van 50 punten

De gebruiker voert zelf een kenteken in, waarna alle info van het bijbehorende voertuig wordt opgehaald. Je hebt de volgende url nodig:

https://opendata.rdw.nl/resource/m9d7-ebf2.json

Documentatie bij de API vind je hier.

De app moet in het scherm de volgende informatie tonen van het opgegeven kenteken:

- Kenteken
- Soort voertuig
- Merk
- Type (handelsbenaming)
- Datum tenaamstelling
- Vervaldatum APK
- Kleur
- Massa rijklaar
- Aantal deuren
- Aantal wielen
- Lengte

- Breedte
- Catalogusprijs

Keuzeopdracht 7.3 – maximaal 40 van 50 punten

De app toont een aflopend gesorteerd overzicht van de 100 meest voorkomende merken en modellen van personenauto's op de Nederlandse snelwegen. Toon de positie, het merk, het model en het aantal.

Keuzeopdracht 7.4 – maximaal 50 van 50 punten

De app toont eerst een lijst met alle verschillende merken personenauto's.

De gebruiker voert zelf een merk en een bouwjaar en maand in, waarna kentekens van auto's van dat merk worden opgehaald die in die maand zijn geregistreerd (datum_eerste_tenaamstelling_in_nederland). Daarna kan de gebruiker een kenteken selecteren, waarna alle detailinformatie wordt getoond (zie hieronder).

Je hebt de volgende url nodig:

https://opendata.rdw.nl/resource/m9d7-ebf2.json

Documentatie bij de API vind je hier.

De app moet in het scherm de volgende informatie tonen van het opgegeven kenteken:

- Kenteken
- Soort voertuig
- Merk
- Type (handelsbenaming)
- Datum tenaamstelling
- Vervaldatum APK
- Kleur
- Massa rijklaar
- Aantal deuren
- Aantal wielen
- Lengte
- Breedte
- Catalogusprijs

Hoofdstuk 9: Internationale Ondersteuning en Lokalisatie

In dit hoofdstuk richten we ons op het toevoegen van internationale ondersteuning en lokalisatie aan je .NET MAUI-applicatie. Door je applicatie aan te passen voor meerdere talen en regio's, kun je een breder publiek bereiken en een betere gebruikerservaring bieden.

9.1 Wat is Lokalisatie?

Lokalisatie is het proces van het aanpassen van je applicatie om verschillende talen, culturele normen, en regio-instellingen te ondersteunen. Dit omvat het vertalen van teksten, aanpassen van datum- en tijdformaten, valuta, en andere cultureel afhankelijke elementen.

9.2 Implementeren van Lokalisatie in .NET MAUI

9.2.1 Voorbereiden van je Project

1. **Voeg resourcelijsten toe**:

- Maak een map genaamd Resources in je project. Binnen deze map, maak submappen aan voor elke taal die je wilt ondersteunen, bijvoorbeeld Resources/Strings/en voor Engels en Resources/Strings/nl voor Nederlands.

2. **Voeg resx-bestanden toe**:

- Voeg een nieuw resx-bestand toe voor elke taal. Voor Engels, voeg een bestand toe genaamd AppResources.en.resx en voor Nederlands AppResources.nl.resx.

3. **Vul de resx-bestanden**:

- Open elk resx-bestand en voeg sleutel-waardeparen toe voor de te vertalen teksten. Bijvoorbeeld:

    - `AppResources.en.resx`:

```
Welcome=Welcome
Hello=Hello
```

- AppResources.nl.resx:

```
Welcome=Welkom
Hello=Hallo
```

9.2.2 Configureren van Lokalisatie in je Applicatie

1. **Configureer de resourcemanager**:

- Voeg een nieuwe klasse toe genaamd LocalizationResourceManager.cs om de resourcemanager te beheren.

```
public class LocalizationResourceManager
{
    private static LocalizationResourceManager _instance;
    private ResourceManager _resourceManager;
    private CultureInfo _currentCulture;

    public static LocalizationResourceManager Instance => _instance ??=
new LocalizationResourceManager();

    public void Init(Type resource, CultureInfo culture)
    {
       _resourceManager = new ResourceManager(resource);
       _currentCulture = culture;
       CultureInfo.CurrentUICulture = _currentCulture;
    }

    public string this[string key] => _resourceManager.GetString(key,
       _currentCulture);

    public void SetCulture(CultureInfo culture)
    {
        _currentCulture = culture;
        CultureInfo.CurrentUICulture = _currentCulture;
    }
}
```

2. **Initialiseer de resourcemanager**:

- Initialiseer de LocalizationResourceManager in MauiProgram.cs.

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            });

        LocalizationResourceManager.Instance.Init(typeof(AppResources),
             CultureInfo.CurrentUICulture);

        return builder.Build();
    }
}
```

3. **Gebruik de gelokaliseerde bronnen in je Views**:

- Pas je XAML-bestanden aan om gelokaliseerde teksten weer te geven.

```
<Label Text="{x:Static
ocal:LocalizationResourceManager.Instance[Welcome]}" />
```

9.2.3 Dynamische Lokalisatie

1. **Mogelijkheid om taal te wijzigen**:

- Voeg functionaliteit toe om de taal van de applicatie dynamisch te wijzigen.

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
    }

    private void OnChangeLanguageButtonClicked(object sender, EventArgs
)
    {
        var newCulture = CultureInfo.CurrentUICulture.Name == "en" ? new
             CultureInfo("nl") : new CultureInfo("en");
        LocalizationResourceManager.Instance.SetCulture(newCulture);
        OnPropertyChanged(nameof(LocalizedWelcome));
    }

    public string LocalizedWelcome =>
       LocalizationResourceManager.Instance["Welcome"];
}
```

2. **Verbind de dynamische tekst aan de UI**:

- Update je XAML-bestand om de dynamische tekst te binden.

```
<Label Text="{Binding LocalizedWelcome, Source={x:Reference
Name=MainPage}}" />
<Button Text="Change Language" Clicked="OnChangeLanguageButtonClicked" />
```

Oefeningen

Oefening 31: Voeg Lokalisatie toe aan je Applicatie

1. **Doel**: Voeg ondersteuning toe voor meerdere talen in je applicatie.

- **Stappen**:
- Maak resx-bestanden aan voor ten minste twee talen.
- Voeg vertalingen toe voor enkele teksten in je applicatie.
- Configureer de resourcemanager om de juiste vertalingen weer te geven.

2. **Controle**: Bouw en voer je applicatie uit om te controleren of de juiste vertalingen worden weergegeven op basis van de ingestelde taal.

Oefening 32: Dynamische Taalwisseling

1. **Doel**: Implementeer functionaliteit om de taal van je applicatie dynamisch te wijzigen.

2. **Stappen**:

- Voeg een knop toe aan je applicatie om de taal te wisselen.
- Implementeer de logica om de huidige cultuur te veranderen en de UI bij te werken.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de taal dynamisch kan worden gewijzigd en de juiste vertalingen worden weergegeven.

Conclusie

In dit hoofdstuk heb je geleerd hoe je internationale ondersteuning en lokalisatie kunt toevoegen aan je .NET MAUI-applicatie. Je hebt gezien hoe je resx-bestanden kunt gebruiken voor het opslaan van vertalingen, hoe je een resourcemanager kunt configureren, en hoe je dynamische taalwisseling kunt implementeren. In de volgende hoofdstukken zullen we verder gaan met het verkennen van toegang tot native API's en het implementeren van platformspecifieke functionaliteiten. Laten we doorgaan en je .NET MAUI-vaardigheden verder ontwikkelen!

Inleveropdacht 8

Opdrachtbeschrijving:

Ontwikkel een mobiele applicatie met .NET MAUI waarin je internationale ondersteuning en lokalisatie implementeert, zoals besproken in hoofdstuk 8. De applicatie moet gebruikers in staat stellen om dynamisch te wisselen tussen verschillende talen, en moet correct functioneren in de gekozen talen.

Scenario:

Je ontwikkelt een eenvoudige takenbeheer-app die wereldwijd gebruikt kan worden. Gebruikers moeten de app in hun eigen taal kunnen gebruiken, met ondersteuning voor zowel Engels als Nederlands. De gebruiker moet tijdens het gebruik van de app kunnen wisselen tussen deze talen.

Eisen:

1. **Voorbereiden van je Project:**
    - Zorg ervoor dat je project correct is voorbereid voor lokalisatie. Dit omvat het opzetten van resourcebestanden voor de verschillende talen (bijv. Resources.resx voor Engels en Resources.nl.resx voor Nederlands).
    - Voeg vertalingen toe voor de belangrijkste teksten in de app, zoals knoppen, labels, en berichten.
2. **Configureren van Lokalisatie in je Applicatie:**
    - Configureer je .NET MAUI-applicatie om automatisch de juiste taal te selecteren op basis van de systeeminstellingen van het apparaat.
    - Zorg ervoor dat de app in staat is om verschillende talen correct weer te geven, inclusief speciale tekens en leestekens die specifiek zijn voor die talen.
3. **Dynamische Lokalisatie:**
    - Implementeer een functionaliteit waarmee de gebruiker handmatig kan wisselen tussen Engels en Nederlands, zonder de app opnieuw te hoeven opstarten.
    - Zorg ervoor dat alle teksten in de app direct worden bijgewerkt wanneer de taal wordt gewijzigd.

Projectvereisten:

- De takenbeheer-app moet minimaal drie hoofdschermen bevatten: een lijst met taken, een scherm om een nieuwe taak toe te voegen, en een instellingenpagina waar de taal kan worden gewijzigd.
- Zorg ervoor dat alle UI-elementen correct zijn gelokaliseerd in zowel Engels als Nederlands.
- Implementeer een eenvoudige gebruikersinterface voor het wisselen van de taal, bijvoorbeeld via een dropdown-menu of knoppen op de instellingenpagina.

Beoordelingscriteria:

- **Lokalisatie:** Zijn alle tekstuele elementen correct gelokaliseerd in de twee talen? Is de lokalisatie consistent en duidelijk?
- **Dynamische Taalwisseling:** Functioneert de dynamische taalwisseling naar behoren? Wordt de taal van de app direct en zonder fouten gewijzigd?
- **Gebruikerservaring:** Hoe gemakkelijk is het voor de gebruiker om van taal te wisselen? Is de interface intuïtief?
- **Codekwaliteit:** Is de code schoon en onderhoudbaar? Zijn de resourcebestanden goed georganiseerd?

Op te leveren:

1. De broncode van de applicatie, inclusief resourcebestanden voor de twee talen.
2. Een korte beschrijving van hoe je de lokalisatie hebt geïmplementeerd en welke uitdagingen je bent tegengekomen.
3. Screenshots of een korte video waarin je de taalwisseling in de app demonstreert.

Hoofdstuk 10: Integratie van Externe Bibliotheken

In dit hoofdstuk richten we ons op het integreren van externe bibliotheken in je .NET MAUI-applicatie. Externe bibliotheken kunnen je helpen bij het snel toevoegen van geavanceerde functionaliteiten zonder dat je alles vanaf nul hoeft te implementeren.

10.1 Wat zijn Externe Bibliotheken?

Externe bibliotheken zijn kant-en-klare codepakketten die door andere ontwikkelaars zijn geschreven en die je kunt hergebruiken in je eigen projecten. Deze bibliotheken kunnen variëren van eenvoudige hulpprogramma's tot complexe frameworks en kunnen je ontwikkelingstijd aanzienlijk verkorten.

10.2 Installeren van Externe Bibliotheken

Het integreren van externe bibliotheken in een .NET MAUI-project is eenvoudig met behulp van NuGet, een pakketbeheerder voor .NET. Je kunt NuGet-pakketten toevoegen via Visual Studio of de .NET CLI.

10.2.1 Installatie via Visual Studio

1. **Open het NuGet-pakketbeheer**:

- Rechtermuisklik op je project in de Solution Explorer.
- Selecteer "Manage NuGet Packages".

2. **Zoek en Installeer het Pakket**:

- Zoek naar het gewenste pakket in het NuGet-pakketbeheer.
- Klik op "Install" om het pakket toe te voegen aan je project.

10.2.2 Installatie via de .NET CLI

Je kunt ook NuGet-pakketten installeren via de .NET CLI.

```
dotnet add package [Pakketnaam]
```

10.3 Voorbeeld: Integratie van SQLite voor Lokale Dataopslag

Een veelvoorkomende use-case is het gebruik van een lokale database voor dataopslag. SQLite is een lichtgewicht, ingebedde database die perfect geschikt is voor mobiele en desktopapplicaties.

10.3.1 Installeren van het SQLite NuGet-pakket

Installeer het `sqlite-net-pcl` NuGet-pakket, een populaire SQLite ORM-bibliotheek voor .NET.

```
dotnet add package sqlite-net-pcl
```

10.3.2 Configureren van SQLite in je Project

1. **Maak een Datamodel**:

- Definieer een klasse die de structuur van je data vertegenwoordigt.

```
public class Note
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }
    public string Text { get; set; }
    public DateTime Date { get; set; }
}
```

2. **Maak een Database Service**:

- Voeg een klasse toe die verantwoordelijk is voor de databaseoperaties.

```
public class DatabaseService
{
    private readonly SQLiteAsyncConnection _database;

    public DatabaseService(string dbPath)
    {
        _database = new SQLiteAsyncConnection(dbPath);
        _database.CreateTableAsync<Note>().Wait();
    }

    public Task<List<Note>> GetNotesAsync()
    {
        return _database.Table<Note>().ToListAsync();
    }

    public Task<Note> GetNoteAsync(int id)
    {
        return _database.Table<Note>().Where(i => i.Id == id).FirstOrDefaultAsync();
    }

    public Task<int> SaveNoteAsync(Note note)
    {
        if (note.Id != 0)
        {
            return _database.UpdateAsync(note);
        }
        else
        {
            return _database.InsertAsync(note);
        }
    }

    public Task<int> DeleteNoteAsync(Note note)
    {
        return _database.DeleteAsync(note);
    }
}
```

3. **Initialiseer de Database Service**:

- Voeg de initialisatiecode toe aan MauiProgram.cs.

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            });

        // Database pad configureren
        string dbPath = Path.Combine(FileSystem.AppDataDirectory, "notes.db3");
        builder.Services.AddSingleton<DatabaseService>(s => new DatabaseService(dbPath));

        return builder.Build();
    }
}
```

4. **Gebruik de Database Service in je Pagina's**:

- Voeg code toe om de database service te gebruiken in je pagina's.

```
public partial class MainPage : ContentPage
{
    private readonly DatabaseService _databaseService;

    public MainPage(DatabaseService databaseService)
    {
        InitializeComponent();
        _databaseService = databaseService;
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        var notes = await _databaseService.GetNotesAsync();
        // Toon de notities in de gebruikersinterface
    }

    private async void OnSaveButtonClicked(object sender, EventArgs e)
    {
        var note = new Note
        {
            Text = "Test Note",
            Date = DateTime.Now
        };
        await _databaseService.SaveNoteAsync(note);
        // Update de gebruikersinterface
    }
}
```

Oefeningen

Oefening 33: Integreer een Externe Bibliotheek

1. **Doel**: Voeg een externe bibliotheek toe aan je .NET MAUI-project en gebruik deze in je applicatie.

- **Stappen**:
- Installeer een NuGet-pakket van je keuze.
- Configureer de bibliotheek in je project.
- Implementeer een eenvoudige functionaliteit die gebruikmaakt van de bibliotheek.

2. **Controle**: Bouw en voer je applicatie uit om te controleren of de integratie correct werkt.

Oefening 34: Gebruik SQLite voor Lokale Dataopslag

1. **Doel**: Implementeer lokale dataopslag in je applicatie met behulp van SQLite.
2. **Stappen**:

- Installeer het sqlite-net-pcl NuGet-pakket.
- Definieer een datamodel en maak een database service.
- Initialiseer de database service en gebruik deze in je pagina's om data op te slaan en op te halen.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de data correct wordt opgeslagen en opgehaald.

Conclusie

In dit hoofdstuk heb je geleerd hoe je externe bibliotheken kunt integreren in je .NET MAUI-applicatie. Door gebruik te maken van NuGet-pakketten kun je snel geavanceerde functionaliteiten toevoegen aan je project. Je hebt ook gezien hoe je SQLite kunt gebruiken voor lokale dataopslag. In de volgende hoofdstukken zullen we verder gaan met het verkennen van geavanceerde navigatiepatronen en het implementeren van pushnotificaties. Laten we doorgaan en je .NET MAUI-vaardigheden verder ontwikkelen!

Inleveropdracht 9

Opdrachtbeschrijving:

Ontwikkel een mobiele applicatie met .NET MAUI waarin je ten minste één externe bibliotheek integreert voor een specifieke functionaliteit. Je zult de gekozen bibliotheek installeren, configureren en implementeren in je app om deze functionaliteit toe te voegen.

Scenario:

Je ontwikkelt een eenvoudige notitie-app waarin gebruikers hun notities kunnen opslaan, doorzoeken en synchroniseren. De app moet gebruik maken van een externe bibliotheek voor het opslaan van notities in een lokale database, met optionele synchronisatiefunctionaliteit.

Eisen:

1. **Installatie van Externe Bibliotheken:**
    - Kies een externe bibliotheek die relevant is voor de functionaliteit van je app. Bijvoorbeeld, gebruik SQLite voor lokale dataopslag of een andere bibliotheek voor synchronisatie met een cloudservice.
    - Installeer de gekozen bibliotheek via Visual Studio of de .NET CLI en zorg ervoor dat deze correct is geïntegreerd in je project.
2. **Integratie van SQLite voor Lokale Dataopslag:**
    - Implementeer lokale dataopslag in je notitie-app met behulp van SQLite. Zorg ervoor dat gebruikers notities kunnen aanmaken, bewerken, verwijderen en doorzoeken.
    - Maak gebruik van een data access layer (DAL) om de interactie met de database te beheren.
3. **Optionele Functionaliteit:**
    - Integreer, indien van toepassing, een tweede externe bibliotheek om synchronisatie met een cloudservice te realiseren. Dit kan bijvoorbeeld een REST API-client zijn om notities naar een cloudserver te sturen en op te halen.
    - Zorg ervoor dat de app robuust omgaat met netwerkverbindingsproblemen en dat de synchronisatie correct functioneert.

Projectvereisten:

- De notitie-app moet minimaal de volgende functionaliteiten hebben: notities aanmaken, bewerken, verwijderen, en doorzoeken.
- De app moet gebruik maken van SQLite voor de lokale opslag van notities, en de bibliotheek moet correct zijn geïntegreerd en gebruikt.
- Indien er een synchronisatiefunctie is geïmplementeerd, moet deze naadloos werken, met duidelijke feedback aan de gebruiker over de synchronisatiestatus.

Beoordelingscriteria:

- **Integratie van Externe Bibliotheken:** Is de bibliotheek correct geïnstalleerd en geïntegreerd? Wordt de bibliotheek effectief gebruikt om de benodigde functionaliteit te bieden?
- **Dataopslag:** Functioneert de lokale dataopslag zoals verwacht? Zijn CRUD-operaties (Create, Read, Update, Delete) correct geïmplementeerd?
- **Optionele Synchronisatie:** Indien geïmplementeerd, werkt de synchronisatiefunctie goed en zonder fouten? Is de synchronisatie robuust en gebruiksvriendelijk?
- **Codekwaliteit:** Is de code schoon, goed gestructureerd, en zijn de externe bibliotheken op een logische en onderhoudbare manier geïntegreerd?

Op te leveren:

1. De broncode van de applicatie, inclusief de integratie van de externe bibliotheek(en).
2. Een korte beschrijving van de externe bibliotheken die je hebt gekozen, waarom je ze hebt gekozen, en hoe je ze hebt geïmplementeerd.
3. Screenshots of een korte video waarin je de belangrijkste functionaliteiten van de app demonstreert.

Hoofdstuk 11: Push Notifications in .NET MAUI

In dit hoofdstuk gaan we dieper in op het implementeren van pushnotificaties in je .NET MAUI-applicatie. Pushnotificaties zijn een krachtig middel om gebruikers te betrekken en te informeren over belangrijke updates, zelfs als de applicatie niet actief is. We zullen bespreken wat pushnotificaties zijn, hoe je een pushnotificatiedienst kunt instellen, en hoe je deze integreert in een .NET MAUI-applicatie voor Android en Windows.

11.1 Wat zijn Push Notifications?

Pushnotificaties zijn berichten die vanaf een server naar de apparaten van gebruikers worden verzonden. Ze worden meestal gebruikt om gebruikers te informeren over nieuwe inhoud, belangrijke updates of andere meldingen die onmiddellijke aandacht vereisen.

11.2 Pushnotificatiediensten

Er zijn verschillende diensten die pushnotificaties ondersteunen, zoals:

- **Firebase Cloud Messaging (FCM)**: Een gratis dienst van Google voor het verzenden van meldingen naar Android- en iOS-apparaten.
- **Azure Notification Hubs**: Een schaalbare dienst van Microsoft voor het verzenden van pushnotificaties naar meerdere platformen.
- **Apple Push Notification Service (APNs)**: De dienst van Apple voor het verzenden van meldingen naar iOS-apparaten.

In dit hoofdstuk focussen we ons op Firebase Cloud Messaging (FCM) vanwege de brede ondersteuning en eenvoud in gebruik.

11.3 Instellen van Firebase Cloud Messaging

11.3.1 Een Firebase Project Aanmaken

1. **Ga naar de Firebase Console**:

- Navigeer naar Firebase Console.
- Log in met je Google-account.

2. **Maak een Nieuw Project Aan**:

- Klik op "Add project".
- Volg de stappen om een nieuw project te creëren.

3. **Configureer de Projectinstellingen**:

- Ga naar Project Settings.
- Noteer de Server Key en Sender ID onder de tab "Cloud Messaging".

11.3.2 Android-configuratie

1. **Firebase Configuratiebestand Toevoegen**:

- Download het google-services.json bestand vanuit de Firebase Console.
- Plaats dit bestand in de app map van je Android project (Platforms/Android/app).

2. **Build Configuration Updaten**:

- Open build.gradle en voeg de volgende regels toe:

gradle
```
apply plugin: 'com.google.gms.google-services'

// In de dependencies sectie
dependencies {
    implementation 'com.google.firebase:firebase-messaging:20.2.4'
}
```

11.3.3 Windows-configuratie

Windows ondersteunt pushnotificaties via het Windows Notification Service (WNS). Je kunt Azure Notification Hubs gebruiken om pushnotificaties naar Windows-apparaten te sturen.

1. **Azure Notification Hubs Instellen**:

- Maak een Notification Hub aan in de Azure Portal.
- Configureer de hub met de vereiste WNS-instellingen.

11.4 Pushnotificaties Integreren in .NET MAUI

11.4.1 Android Implementatie

1. **Firebase Initialiseren in MainActivity**:

```
using Android.App;
using Android.Content.PM;
using Android.Runtime;
using Android.OS;
using Firebase;

[Activity(Label = "MyApp", Icon = "@mipmap/icon", Theme =
"@style/MainTheme", MainLauncher = true, ConfigurationChanges =
ConfigChanges.ScreenSize | ConfigChanges.Orientation)]
public class MainActivity : global::Microsoft.Maui.MauiAppCompatActivity
{
    protected override void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);

        FirebaseApp.InitializeApp(this);
    }
}
```

2. **Firebase Messaging Service Toevoegen**:

```
using Firebase.Messaging;
using Android.Util;

[Service]
[IntentFilter(new[] { "com.google.firebase.MESSAGING_EVENT" })]
public class MyFirebaseMessagingService : FirebaseMessagingService
{
    const string TAG = "MyFirebaseMsgService";

    public override void OnMessageReceived(RemoteMessage message)
    {
        base.OnMessageReceived(message);
        Log.Debug(TAG, "From: " + message.From);
        Log.Debug(TAG, "Notification Message Body: " +
message.GetNotification().Body);

        // Toon notificatie aan gebruiker
        SendNotification(message.GetNotification().Body);
    }

    void SendNotification(string messageBody)
    {
        var notificationManager = NotificationManager.FromContext(this);
        var notification = new Notification.Builder(this)
            .SetContentTitle("New Message")
            .SetContentText(messageBody)
            .SetSmallIcon(Resource.Drawable.ic_stat_ic_notification)
            .Build();

        notificationManager.Notify(0, notification);
    }
}
```

11.4.2 Windows Implementatie

1. **NuGet Pakket Installeren**:
   - Installeer `Microsoft.Azure.NotificationHubs` in je project.
2. **Configuratie van de Notification Hub**:

csharp
Code kopiëren

```
using Microsoft.Azure.NotificationHubs;

public class NotificationService
{
    private static NotificationHubClient hub;

    public NotificationService()
    {
        hub =
NotificationHubClient.CreateClientFromConnectionString("<connection
string>", "<hub name>");
    }

    public async Task SendNotificationAsync(string message)
    {
        var notif = new WindowsNotification(message);
        await hub.SendNotificationAsync(notif);
    }
}
```

3. **Verwerken van Meldingen**:

Gebruik de `PushNotificationReceived`-event in je `App.xaml.cs`:

```
public partial class App : Application
{
    public App()
    {
        InitializeComponent();

        // Abonneer je op meldingen
        PushNotificationManager.NotificationReceived +=
OnPushNotificationReceived;
    }

    private void OnPushNotificationReceived(object sender,
PushNotificationEventArgs e)
    {
        // Verwerk de melding
        MainPage.DisplayAlert("Push Notification", e.Message, "OK");
    }
}
```

Oefeningen

Oefening 35: Configureren van Firebase Cloud Messaging

1. **Doel**: Stel een Firebase-project in en configureer je .NET MAUI Android-app voor pushnotificaties.
2. **Stappen**:

- Maak een Firebase-project aan en voeg je Android-app toe.
- Download google-services.json en voeg deze toe aan je Android-project.
- Configureer je Android-project voor Firebase.

3. **Controle**: Bouw en voer je applicatie uit om te controleren of de Firebase-configuratie correct is ingesteld.

Oefening 36: Implementeer Pushnotificaties voor Android

1. **Doel**: Voeg pushnotificaties toe aan je .NET MAUI Android-app.
2. **Stappen**:

- Voeg een Firebase Messaging Service toe aan je project.
- Configureer de service om meldingen te ontvangen en weer te geven.

3. **Controle**: Bouw en voer je applicatie uit en stuur een testmelding vanuit de Firebase Console om te controleren of de pushnotificaties correct werken.

Oefening 37: Implementeer Pushnotificaties voor Windows

1. **Doel**: Voeg pushnotificaties toe aan je .NET MAUI Windows-app.
2. **Stappen**:

- Stel Azure Notification Hubs in en configureer deze voor Windows-notificaties.
- Voeg de benodigde code toe om meldingen te verwerken en weer te geven.

3. **Controle**: Bouw en voer je applicatie uit en stuur een testmelding vanuit Azure Notification Hubs om te controleren of de pushnotificaties correct werken.

Conclusie

In dit hoofdstuk heb je geleerd hoe je pushnotificaties kunt implementeren in je .NET MAUI-applicatie voor zowel Android als Windows. Pushnotificaties zijn een essentieel onderdeel van moderne applicaties en kunnen helpen bij het verbeteren van de gebruikersbetrokkenheid en het informeren van gebruikers over belangrijke gebeurtenissen.

Inleveropdracht 10

Opdrachtbeschrijving:

Ontwikkel een mobiele applicatie met .NET MAUI waarin je pushnotificaties implementeert. De app moet gebruik maken van Firebase Cloud Messaging (FCM) om pushnotificaties te verzenden en te ontvangen, en je moet de implementatie voor zowel Android als Windows voorzien.

Scenario:

Je ontwikkelt een nieuwsapp waarin gebruikers op de hoogte worden gehouden van het laatste nieuws via pushnotificaties. Gebruikers kunnen meldingen ontvangen voor verschillende nieuwssecties, zoals "Sport", "Technologie", en "Entertainment". De app moet meldingen kunnen ontvangen, en deze correct weergeven afhankelijk van de platforminstellingen.

Eisen:

1. **Instellen van Firebase Cloud Messaging (FCM):**
    - Maak een Firebase-project aan en configureer FCM voor je .NET MAUI-app.
    - Implementeer de Android-configuratie en de Windows-configuratie om pushnotificaties te kunnen ontvangen.
2. **Android Implementatie:**
    - Configureer de app om pushnotificaties te ontvangen op Android-apparaten.
    - Zorg ervoor dat de app correct omgaat met notificaties, zowel wanneer de app actief is als wanneer deze op de achtergrond draait.
    - Implementeer een logica die ervoor zorgt dat de ontvangen notificaties correct worden weergegeven in het notificatiecentrum van Android.
3. **Windows Implementatie:**
    - Configureer de app om pushnotificaties te ontvangen op Windows-apparaten.
    - Zorg ervoor dat notificaties correct worden weergegeven, zowel in de app als in het Windows-notificatiecentrum.
    - Implementeer logica om binnenkomende meldingen te verwerken, zoals het bijwerken van de UI of het openen van specifieke inhoud in de app wanneer de gebruiker op een notificatie klikt.
4. **Gebruikersinteractie met Notificaties:**
    - Zorg ervoor dat de app op een logische manier omgaat met gebruikersinteracties met notificaties. Bijvoorbeeld, wanneer een gebruiker op een notificatie klikt, moet de app naar de juiste nieuwssectie of artikel navigeren.
5. **Optionele Functionaliteit:**
    - Implementeer een instellingenpagina waar gebruikers kunnen aangeven voor welke nieuwssecties ze meldingen willen ontvangen.

    - Voeg een eenvoudige logging- of monitoringsfunctie toe om bij te houden wanneer en hoe vaak pushnotificaties worden ontvangen en verwerkt.

Projectvereisten:

- De nieuwsapp moet pushnotificaties ontvangen op zowel Android als Windows en deze correct weergeven, afhankelijk van de platforminstellingen.
- Gebruikers moeten interacties met notificaties kunnen aangaan, zoals het openen van de juiste nieuwssectie of een specifiek artikel vanuit de notificatie.
- (Optioneel) Gebruikers kunnen hun notificatievoorkeuren instellen via een instellingenpagina.

Beoordelingscriteria:

- **FCM Configuratie:** Is Firebase correct geconfigureerd, en kan de app pushnotificaties ontvangen op zowel Android als Windows?
- **Android Implementatie:** Worden pushnotificaties correct ontvangen en verwerkt op Android, zowel in de voorgrond als op de achtergrond?
- **Windows Implementatie:** Worden pushnotificaties correct ontvangen en verwerkt op Windows, zowel in de voorgrond als op de achtergrond?
- **Gebruikersinteractie:** Werkt de gebruikersinteractie met notificaties zoals verwacht? Navigeert de app correct naar de juiste inhoud?
- **Codekwaliteit:** Is de code schoon, goed georganiseerd, en zijn pushnotificaties op een logische en onderhoudbare manier geïmplementeerd?

Op te leveren:

1. De broncode van de applicatie, inclusief de volledige implementatie van pushnotificaties voor Android en Windows.
2. Een korte beschrijving van de implementatiestappen, inclusief eventuele uitdagingen en hoe je deze hebt opgelost.
3. Screenshots of een korte video waarin je de werking van pushnotificaties in de app demonstreert.

Hoofdstuk 12: Eindproject

Nu je alle basis- en geavanceerde aspecten van .NET MAUI hebt geleerd, is het tijd om je kennis in de praktijk te brengen met een groot eindproject. In dit project zul je een complete .NET MAUI-applicatie ontwikkelen die meerdere platformen ondersteunt en gebruik maakt van de diverse functies die we hebben behandeld. Dit project geeft je de kans om alles wat je hebt geleerd samen te brengen en een professionele applicatie te creëren.

Projectopdracht: Een Complete .NET MAUI Applicatie

Doel: Ontwikkel een complete .NET MAUI-applicatie die zowel op Windows als Android werkt. De applicatie moet gebruik maken van de volgende functies:

- Meerdere pagina's en navigatie
- Data binding en MVVM-patroon
- Gegevensopslag (local database)
- Toegang tot apparaat functies (zoals camera of locatie)
- Netwerkcommunicatie (bijvoorbeeld API-aanroepen)
- Pushnotificaties

Stappen

1. **Kies een Casus**: Kies een scenario of casus voor je applicatie. Enkele voorbeelden:

- Een takenbeheer-app
- Een fitness-tracker
- Een receptenboek
- Een evenementenplanner

2. **Maak een Projectplan**: Schrijf een projectplan waarin je de functionaliteiten van je applicatie beschrijft. Denk aan de volgende onderdelen:

- Overzicht van de applicatie
- Doelgroep
- Functionele vereisten
- Technische vereisten
- Wireframes of schetsen van de UI

3. **Ontwikkel de Applicatie**:

- **Instellen van de Omgeving**: Configureer je ontwikkelomgeving voor zowel Windows als Android.
- **Maak de Basisstructuur**: Implementeer de basisnavigatie en paginalayouts.
- **Data Binding en MVVM**: Gebruik MVVM voor data binding en scheiding van logica.
- **Gegevensopslag**: Implementeer lokale gegevensopslag met SQLite.
- **Apparaatfuncties**: Integreer functies zoals camera, locatie, etc.
- **Netwerkcommunicatie**: Voeg API-aanroepen toe voor het ophalen en verzenden van gegevens.

- **Pushnotificaties**: Configureer en implementeer pushnotificaties voor Android en Windows.

4. **Testen en Debuggen**: Test je applicatie uitvoerig op zowel Windows als Android om ervoor te zorgen dat alle functies correct werken en er geen bugs zijn.
5. **Documentatie**: Schrijf een gebruikershandleiding en technische documentatie voor je applicatie.
6. **Presentatie**: Bereid een presentatie voor waarin je je applicatie demonstreert en uitlegt welke technieken en oplossingen je hebt gebruikt.

Beoordelingsmatrix

Je eindproject wordt beoordeeld op de volgende criteria:

Criteria	Uitstekend (5)	Goed (4)	Voldoende (3)	Onvoldoende (2)	Slecht (1)
Projectplan	Zeer gedetailleerd en goed doordacht	Duidelijk en goed georganiseerd	Basisbeschrijving met enkele details	Incompleet of slecht georganiseerd	Onvoldoende of niet ingeleverd
Functionaliteit	Alle vereiste functies werken foutloos en efficiënt	De meeste vereiste functies werken goed	Enkele functies werken, maar er zijn enkele problemen	Veel functies werken niet of zijn buggy	Meeste functies werken niet
Navigatie en UI	Uitstekende UI/UX en intuïtieve navigatie	Goede UI/UX en navigatie	Redelijke UI/UX en navigatie	Slechte UI/UX of verwarrende navigatie	Zeer slechte UI/UX en navigatie
Data Binding en MVVM	Correct en volledig geïmplementeerd	Goed geïmplementeerd met enkele kleine fouten	Basisimplementatie met meerdere fouten	Slechte implementatie met veel fouten	Niet geïmplementeerd
Gegevensopslag	Volledig functioneel en goed geïntegreerd	Functioneel met enkele kleine problemen	Basisfunctionaliteit, maar niet optimaal	Onvoldoende functionaliteit	Niet geïmplementeerd
Apparaat functies	Volledig en foutloos geïmplementeerd	Goed geïmplementeerd met enkele fouten	Basisfunctionaliteit met meerdere fouten	Slechte of incomplete implementatie	Niet geïmplementeerd

Criteria	Uitstekend (5)	Goed (4)	Voldoende (3)	Onvoldoende (2)	Slecht (1)
Netwerkcommunicatie	Volledig en foutloos geïmplementeerd	Goed geïmplementeerd met enkele kleine fouten	Basisfunctionaliteit met meerdere fouten	Slechte of incomplete implementatie	Niet geïmplementeerd
Pushnotificaties	Volledig en foutloos geïmplementeerd	Goed geïmplementeerd met enkele kleine fouten	Basisfunctionaliteit met meerdere fouten	Slechte of incomplete implementatie	Niet geïmplementeerd
Testen en Debuggen	Uitgebreid getest en volledig bugvrij	Goed getest met enkele kleine bugs	Redelijk getest met meerdere bugs	Slecht getest met veel bugs	Niet getest of veel bugs
Documentatie	Zeer gedetailleerd en volledig	Goed gedocumenteerd	Basisdocumentatie	Incomplete of slechte documentatie	Geen documentatie
Presentatie	Zeer professioneel en duidelijk	Goede en duidelijke presentatie	Redelijke presentatie	Onvoldoende of verwarrende presentatie	Slechte of geen presentatie

Conclusie

Dit eindproject biedt een geweldige kans om je vaardigheden in .NET MAUI te demonstreren en een complete, professionele applicatie te ontwikkelen. Gebruik je kennis van navigatie, data binding, apparaat functies, netwerkcommunicatie, en pushnotificaties om een project te creëren dat zowel functioneel als gebruiksvriendelijk is. Veel succes!